

Fractal Intelligence

Conceptual Decomposition as Problem-Solving Infrastructure

Henrik Westerberg

henrik.westerberg@emergentwisdom.org

April 7, 2026

Revised: August 25, 2026

Abstract

Human decompositions often inherit path-dependent conventions. This paper instead asks what a problem is made of: concepts that may transfer across domains, rather than solution steps. Re-deriving its structure may surface inherited assumptions and expose dimensions, categories, and candidate mechanisms that conventional framing leaves unseen. *Fractal intelligence* names a form of cumulative intelligence in which bounded, concept-defined units recursively decompose and reframe problems, compose child work into higher-level capabilities, and preserve evaluated structures for reuse or revision across problems, domains, and scales.

A root Solver bounds the problem space. A problem ascends through adjacent genus-and-differentia specializations to that root, searching for parents and siblings; it then retraces the route before decomposing the specific capability into four-tested constituents. Specialization locates kinds; composition binds differentiated contributions. The five-surface Solver Contract makes resulting units recursively composable; a marginal-value rule governs depth; content-addressed Pathway Memory supports revision and reuse. Parent synthesis states how child contributions, routing, integration, and evaluation realize one cell-like outward capability. A model, human, program, or subtree may supply any interior, and one Solver may serve several parents without changing its contract. Typed rejection can request re-decomposition; recurrent validated structures may become reasoning highways.

A 100-problem mandatory-abstraction construction and an alpha reference implementation show that the proposed topology can be instantiated and structurally checked, but neither tests solution quality. The decisive tests are whether fresh conceptual carves outperform equally persistent conventional approaches under matched compute and memory, whether independent carves recover recurring invariants or complementary utility beyond arbitrary divergence, and whether reuse improves subsequent problem solving. The paper presents a testable theory and architecture, not outcome evidence.

Keywords: Fractal Intelligence, Conceptual Decomposition, Multi-Agent Systems, Cognitive Architecture, Solver Contracts

1 Introduction

This paper begins with two simple claims. First, solved reasoning should not evaporate. Most agent systems discard the structure that produced an answer, or preserve only a transcript that later systems must interpret from scratch. An evaluated solution graph—its concepts, route, outputs, gate results, provenance, and outcome—should remain globally addressable, subject to permissions, so the next relevant problem can begin from what has already been worked out.

Second, the inherited way of cutting problems may not be the best one. Current multi-agent frameworks naturally reproduce the task boundaries of human organizations: assess, plan, assign, execute. Those boundaries coordinate people well, but they are path-dependent products of institutions and history. A model trained on human text does not escape them by itself; it inherits the categories as well as the knowledge.

The useful thought experiment is an alien analyst arriving with no business school, ministry, or professional guild to imitate. Asked to understand an economy, it does not begin from our departments or procedures. It asks what an economy is actually made of, moves upward through adjacent abstractions, tests each proposed parent against its siblings, then retraces that route and carves downward from the resulting frame. The thought experiment places an explicit decomposition discipline between inherited convention and the solution while recognizing that any language model remains culturally conditioned.

The four tests developed here—necessity, independence, universality, and completeness—bring together MECE-style partitioning and the Aristotelian search for properties that define every member of a category. They provide one practical discipline, not the only possible method, and do not guarantee a uniquely correct tree. A fresh carve may reveal structure convention missed, or it may re-derive the conventional answer and make its hidden reasoning explicit. The imperative to look is therefore distinct from the empirical claim that looking wins; the latter must be established against equally resourced conventional baselines.

The broader research program distinguishes three complementary motions: *jump farther* by changing the causal premises from which mechanisms are derived, *search wider* by mapping a candidate population and directing generation toward underrepresented mechanism families, and *decompose deeper* by recursively isolating the concepts on which a problem depends. Orthogonal Insight studies one long-range displacement operator [1]; the companion Ontology work studies population-level search state [2]; this paper asks how selective depth, typed composition, and cross-problem persistence can become cumulative intelligence. The studies emphasize different operations and retain different principal objects: a candidate ontology maps proposed interventions, a Solver graph routes concept-defined capabilities, and an evaluated solution graph records what was attempted and checked so useful structure can persist. No experiment has evaluated all three motions together.

Recursive encapsulation is core to this realization: bounded representations and interfaces make lower-level complexity usable at a higher level of cognitive organization. Major evolutionary transitions provide a precedent in which lower-level entities became integrated into higher-level individuals able to function as units at another scale [3, 4]. The Solver architecture reverses the order of boundary assignment for engineering: a root Solver encapsulates the complete problem space of its deployment before particular problems are decomposed within it. Section 2.11 develops this evolutionary precedent and its limits.

There is reason to look. Human expertise is largely implicit: pattern matching, norms, and intuitions accumulated through practice. We rarely write down the *invisible thinking* [5, 6] that produces a conclusion, and we rarely sustain the deliberate, dimension-by-dimension reasoning Kahneman calls System 2 [7]. Machines can sustain it, but only if given structure in which to do

so. Instead of asking “how would a human solve this?”, the architecture asks: “what is this problem made of?”

Consider creating a restaurant. The conventional approach is a sequence of tasks: research the market, secure funding, find a location, hire a chef, design a menu, build out the space, obtain permits, launch marketing, open. This familiar sequence is useful for coordinating execution, but it describes activities rather than reusable conceptual structure.

A conceptual decomposition instead asks two different questions. First: what does it mean to *create* something? This decomposes into intent, form, realization, fit, and viability—dimensions that apply to creating anything, whether a restaurant, a curriculum, or a building. Second: what *is* a restaurant? This decomposes into nourishment, experience, flow, economics, and identity—candidate differentiated dimensions of the thing itself. Each dimension corresponds to a distinct failure mode: nourishment concerns the primary offering, experience the form of service, and flow the coordination of people and resources. Tasks emerge at the intersection of these two decompositions, but the conceptual structure is what persists and transfers.

The distinction is relational rather than grammatical. A task decomposition partitions a chosen route to an outcome into actions, stages, or roles. A conceptual decomposition partitions the capability or problem class itself into functions, constraints, or causal dimensions claimed to be constitutive of a qualifying realization. The same word can occur in either: an action may itself become the subject of conceptual decomposition when treated as a capability, while every concept-defined Solver receives concrete Tasks when invoked. Abstract names and persistence alone do not establish the distinction; what matters is whether the children describe a procedure for producing one result or the proposed structure through which many possible procedures and instances can route.

Once decomposed, each dimension can be assigned to a solver isolated behind its own boundary. Flow—how people and resources move through space and time—shares recurring concerns across a restaurant, a hospital, a transit system, and an irrigation network. Recurring dimensions may therefore become permanent infrastructure—reasoning highways—although the further claim that repeated use sharpens them remains untested.

If that wager pays, an AI assistant could consult something larger than itself: a persistent reasoning network, analogous to the internet’s relation to any single computer. The network would decompose a submitted problem, solve parts in parallel, and reuse structures developed for earlier problems. The assistant would remain responsible for contextual judgment, weighing what returns and overruling it when necessary. Such a system would provide an internet for reasoning rather than retrieval: problems would be submitted for structured resolution rather than answered from model weights alone.

Human civilization already approximates this pattern: a hospital retains diagnostics, surgery, pharmacology, and rehabilitation as specialist structures through which successive patients route. Each case concludes; the hospital’s specialist structures persist. The architecture proposed here makes such structures explicitly composable, content-addressed, and persistent at machine scale.

The rest of this section proceeds in four steps. §1.1 defines fractal intelligence as a general concept, then presents one Solver-based architecture for realizing it and specifies how problems route among its Solvers. §1.2 states the seven theses the paper rests on, each paired with a falsification criterion. §1.3 specifies the six architectural mechanisms that produce them. §1.4 positions the work against its intellectual ancestors and names what this paper adds that existing frameworks do not.

The remainder of the paper separates four evidential layers. Sections 2–4 define the architectural core; Sections 5–8 develop proposed protocols, learning dynamics, and long-horizon implications; Section 9 reports structural construction and implementation probes; and Sections 10–11 distinguish related work, present evidence, and state the unresolved tests. The breadth of an implication is not evidence that its mechanism has been implemented.

1.1 The Concept of Fractal Intelligence

Fractal intelligence is a form of cumulative intelligence in which bounded, concept-defined units recursively decompose and reframe problems, compose child work into higher-level capabilities, and preserve evaluated structures for reuse or revision across problems, domains, and scales.

Fractal intelligence denotes the broader theoretical category. The Solver architecture developed here gives that pattern a concrete form through a root boundary, typed contracts, acceptance gates, Pathway Memory, and a marginal-value rule. These mechanisms specify this paper's realization without exhausting the category.

In this realization, a composite parent coordinates child work behind one contract boundary, so its subtree presents one Solver interface to the caller. A deployment may use one model or many. Because routing is organized by declared conceptual contracts rather than one fixed domain workflow, the architecture is intended to accept problems from many domains; whether its coverage becomes genuinely general is an empirical question. From the outside, it aims to behave as a single general-purpose problem solver. From the inside, it is specialists all the way down.

The foundation of this realization is first-principles thinking. At every node, the decomposition asks the same question: what are the irreducible dimensions of this concept? The term *fractal* is used in an organizational rather than strictly geometric sense: the same relationship recurs at multiple scales. A problem can be decomposed into conceptual subproblems; each subproblem can be decomposed again; and a solved substructure can itself function as a component of a larger one. The five-surface Solver Contract expresses this recurrence uniformly at every level. The resolution increases; the organizational pattern remains.

No individual node needs to be generally intelligent: the intended capability comes from composing specialists through the contract. Typed invocation boundaries can keep the generator's and verifier's working contexts distinct even when both are bound by the same public `AcceptSpec`; the generator need not receive the verifier's private deliberation or verdict. Section 4.6 develops this isolation claim and its evidence.

Candidate nodes arise in two ways: as specialization membranes located along the upward abstraction route, or as compositional Solvers generated by the downward four-test search operator developed in §2. Evaluated solution graphs enter the archive; recurrent, validated, or uniquely useful subgraphs may be promoted into the active serving layer, and Pathway Memory (§5.1) informs later routing under the marginal-value rule (§3.2). The architecture permits alternative search operators, catalogues, and routing policies. The ten protocols are worked examples whose value does not require convergence on a definitive ontology.

1.2 Theses

Seven theses elaborate the two commitments above: retain solved structure, and search for leverage in how problems are decomposed. They do not have the same evidential status; some are architectural claims, others scaling or governance implications. Each is paired with a failure condition. The decisive empirical program has three parts: whether conceptual decomposition beats equally persistent conventional solving at matched compute, whether independently derived carves reveal recurring invariants or complementary utility beyond chance, and whether reuse improves subsequent problem solving. Together, these tests separate architectural feasibility from claims of superior performance, cross-carve invariance, and cumulative benefit.

Thesis 1: Solved structure should persist, and conceptual structure is the candidate unit of transfer (§2, §3.3, §6.4–§6.10). An evaluated solution graph should remain available rather than dissolve after execution. Because task plans can also be cached, memory alone does not establish the distinctive thesis: the empirical bet is that conceptual decomposition—what a problem is *made of*, not merely what to do next—produces units that transfer farther across domains than equally persistent task traces. The protocols instantiate candidate units of transfer, while the construction artifact’s prompt-conditioned, structurally recorded reuse (§9) does not establish superiority. *Falsification*: give concept-decomposed and task-decomposed systems equal global memory, compute, and outcome feedback. If task traces match conceptual graphs in cross-domain reuse and solution quality without implicitly reconstructing concepts, conceptual structure is not a superior transfer unit; the separate value of persistence would remain unresolved.

Thesis 2: Additional intelligence is found by zooming deeper into the fractal, up to the point where incremental decomposition cost exceeds incremental cognitive gain (§3.1, §3.2, §3.4). Deeper conceptual decomposition can sharpen an analysis by isolating sub-concepts that a shallow treatment blends together. Where a ground truth exists, this may improve accuracy; for normatively contested concepts, it instead yields a more fully mapped disagreement structure. The marginal-value rule bounds the claim. *Scope-limiting result*: if well-tuned uniform-depth or shallower solvers systematically outperform marginal-value-guided deeper decomposition across a problem class, then depth is not a general source of intelligence for that class and the rule must be reformulated.

Thesis 3: Organizing structure can contribute intelligence beyond the components alone (§4 and §8). Component capability still constrains the whole, but a parent’s Tasks, routing, synthesis, and gates coordinate child work and make gate outcomes locally assignable without a shared loss function. *Falsification*: if untyped, loose coordination among the same components, under matched compute and task distribution, produces equivalent compositional gains and equivalent per-node learning, the structure-contributes-intelligence thesis is false.

Thesis 4: Cognitive isolation enables productive extremes (§4.6). Competing objectives can compromise one another when they share a forward pass or weights, as the Safety Tax illustrates [8]. Contract-bounded Solvers instead separate cognitive modes so generation and verification need not soften toward one another. *Falsification*: if a single monolithic model, trained with equivalent compute and without hidden modular isolation, matches the composition’s capability on tasks requiring genuinely competing modes, the productive-extremes thesis is false.

Thesis 5: Typed rejection turns failure into a structural reframing signal (§4.1 and §4.6). A rejection can name a failed structural relation and ask the upstream Solver to change its frame rather than merely produce another answer. The companion study documents an analogous precursor: curator rejection with mechanism-level redirection, followed by a retry that changed the causal relation [2]. It did not isolate that effect or include a matched monolithic baseline. *Falsification*: if an equally resourced retry protocol receiving the same history and pass–fail outcome, but no typed structural diagnosis, matches both the frequency and utility of relation-changing reframes and the resulting outcomes, the stronger reframing-over-retry claim fails.

Thesis 6: Evaluation is itself a Solver, and the apparent regress terminates (§4.1 and §6.10). If gates do cognitive work, evaluation is a cognitive task and every evaluator needs its own evaluator. The Solver Contract terminates this regress at two ends: mechanically decidable checks at the bottom (schema checks, hash matches, test vectors), whose verdicts require no language-model judgment even though their specifications and implementations can still be wrong; and the proposed OutcomeArbiterSolver at the top, grounded in human judgment on open-ended problems and in objective checks on verifiable ones. Evaluation therefore terminates in either mechanically decidable criteria or explicitly identified human judgment. *Failure condition*: if evaluator Solvers cannot be grounded more reliably than monolithic evaluators on matched open-ended and verifiable tasks, then this termination strategy is not an advantage over simpler evaluation architectures. The philosophical

grounding of normative judgment at the top remains an open problem regardless of this architecture’s choice.

Thesis 7: A machine-scale realization of fractal intelligence could become a social-organizing principle (§7). Machine-speed coordination through typed contracts, content-addressed patterns, and persistent reasoning may admit organizational structures not derived from the constraints of biological minds. The promise is compositional intelligence at unprecedented scale; the risk is replacing slow, legible, contestable structures with fast, opaque, optimized ones [5]. *Scope-limiting result:* if large-scale typed-contract coordination offers no capability, efficiency, or reliability beyond biological coordination and conventional software organizations on matched tasks, the claim reduces to an engineering pattern. Whether such coordination would improve or erode institutional legitimacy is an independent normative question.

1.3 What This Architecture Adds

The theses above are claims about what the architecture may produce. This subsection specifies six concrete mechanisms proposed to produce them—the architectural primitives on which the theses rest. Some are exercised by the mandatory-abstraction construction and experimental reference implementation described in Section 9, while others remain design commitments. The theses define the tests; the mechanisms define the proposed implementation.

Self-similar contracts. Every node, whether orchestrator, specialist, or leaf, presents the five-surface Solver Contract (Section 4), with Manifest and Execute mandatory and the other three surfaces optional and discoverable. A leaf and a compatible thousand-node subtree can therefore present the same contract boundary to the caller. Because the boundary is uniform, a compatible subtree that solved a problem at depth 7 can be imported at depth 2; without this recursive closure, the other mechanisms lose their foundation.

Gates that request conceptual restructuring. Rejection is a typed diagnostic naming the structural failure and asking the upstream Solver to change its approach rather than merely rephrase it. The retirement-system traces in Section 4.6 illustrate an analogous redirection path; they do not implement a Solver Contract gate or isolate the controller’s causal effect from the companion system’s other differences.

Cognitive isolation enabling productive extremes. Cognitive mode is declared in the Manifest and separated by typed invocation boundaries, so each mode can operate without direct access to the others’ invocation context (Thesis 4) [8].

Economic depth governance. The marginal-value rule (Section 3.2) decides at every node whether the expected gain from further decomposition justifies its incremental cost.

Persistent decomposition and learning through reuse. The decomposition crystallizes as a content-addressed pattern importable by hash (Section 4.5): what persists is the anatomy of a competence, not an answer. The routing layer can direct structurally similar problems to a common Solver through the uniform contract, accumulating cross-domain signal and forming reasoning highways (Section 5.2).

Revisable typed topology. In the proposed architecture, reviewed evidence can insert a higher parent, retire a shortcut, split a Solver doing two conceptual jobs, or merge duplicate boundaries. Specialization and composition edges retain different meanings, while immutable patches preserve what changed. The construction artifact demonstrates graph construction and higher-parent insertion, not autonomous feedback-driven topology learning; its behavior is reported in Section 9.2.

Table 1: Illustrative role-based task pipeline vs. the proposed Solver Tree.

	Role-Based Task Pipeline	Solver Tree
Organizing principle	Actions, stages, or roles in a chosen route	Constitutive capability or dimension (tasks route through it)
Identity basis	Person/Role (the “who”)	Concept/Interface (the “what”)
Topology	Usually specified around a workflow	May adapt through observed use
Boundaries	Often natural-language handoffs	Typed, gated (reject + diagnosis)
Depth allocation	Often stage-based	Selective (marginal-value rule)
Failure signal	Often reported at task level	Checked at seams; detected failures may request reframe
Cognitive mode	Usually attached to task roles	Declared; separated by typed invocation
Output	An answer	An answer + reusable structure
Persistence	Often task-local or trace-based	Evaluated structure retained; recurrent patterns addressable

1.4 Contributions

This paper’s central contribution is the claim that conceptual decomposition can become cumulative reasoning infrastructure: problems are located through mandatory upward abstraction, decomposed recursively downward into concept-defined Solvers, selectively deepened under a marginal-value rule, and retained as revisable evaluated structures that later problems can reuse. Its components have ancestors in means–ends analysis, division of cognitive labor, hierarchical and modular decomposition, distributed cognition, and cybernetic regulation [9, 10, 11, 12, 13, 14]; self-similar organization [15, 16, 17]; contract coordination [18, 19, 20, 21]; morphological analysis [22]; and persistent workflow reuse [23]. What is distinctive is the proposed combination of mandatory upward abstraction, recursive downward decomposition, contract-bounded composition, structural reframing, and accumulated reuse. Section 10 develops the comparisons.

The four tests, and the universality test in particular, are intended to produce candidates whose relevance survives changes in concrete task and domain: they search for axes that recur across the range of instances currently claimed for the parent capability. That range is provisional and may widen, narrow, or be reframed. Independent carves may therefore recover common sub-concepts, but convergence is a prediction rather than a guarantee and is not the only possible success. The empirical test must measure both recurring invariants and whether divergent carves add utility beyond arbitrary divergence. In the mandatory-abstraction construction, reuse occurs inside one theory-conditioned graph; it demonstrates structurally accepted routing and higher-parent insertion, not independent discovery or highway formation.

Within the proposed Solver architecture, two complementary frameworks operationalize one route toward that idea: the **Theory of Depth** (Section 3) governs when and where to decompose, while the **Solver Contract** (Section 4) specifies how concept-defined Solvers compose through typed surfaces and gates. Orthogonal Insight and ontology-governed expansion are related search methods in the broader program, compatible with this architecture but neither definitions of fractal intelligence nor empirical results of the mandatory-abstraction construction reported here. Consequences of the proposed substrate include productive extremes (Section 4.6), localized learning (Section 5), and a possible cognitive commons (Section 7).

The contributions occupy different evidential levels: the *formalization* is specified and partly instantiated; the *protocols* are design proposals; and the *unifying claim* that conceptual decomposition enables cross-domain reuse remains an empirical hypothesis for the reproduction study in Section 11.

2 The Conceptual Decomposition Algorithm

This section makes one first-principles decomposition operator explicit enough to be executed by large language models. The four tests and pseudocode are a starting discipline rather than a complete specification of creative reasoning; practical implementations may use judgment, iteration, hybrid collaboration, or alternative search operators. Three author-designed examples illustrate the intended cross-domain candidates; machine-generated decompositions appear in the canonical mandatory-abstraction construction in Section 9. The architecture turns first-principles decomposition from an individual reasoning practice into an explicit, inspectable coordination procedure without assuming that this procedure exhausts the possible good carves.

2.1 Concepts, Capabilities, Tasks, and Context

A *concept* is a candidate invariant used to organize the problem space: either a kind located through specialization or a constitutive dimension linked through composition. A *capability* is the outward function through which that concept becomes operational. A *Solver* is the contract boundary around an implementation of the capability, whereas a *Task* is one invocation carrying particular inputs, constraints, budget, and acceptance criteria. *Context* comprises the invocation-specific and parent-local conditions required to apply the capability without silently redefining its shared core. A parent’s *synthesis* states how its children’s differentiated contributions, together with routing and integration, realize the outward capability promised at its own boundary.

The concept–task distinction therefore concerns the relation asserted by a decomposition, not the abstractness of a label. Task decomposition partitions a selected procedure; conceptual decomposition proposes what the target capability or problem class is made of. A reusable task method remains task-organized if it encodes a route, while an apparent action can become a conceptual subject if the system asks what capability that action realizes and what constitutes it. Selecting the same stored node or content-addressed Solver again is *structural reuse*; claiming that competence transferred requires the stronger evidence that its contract remained valid and its use improved or preserved relevant outcomes in the new context.

The four tests interrogate a *provisional claim frame*: the outward capability being promised, the purpose for which it is being decomposed, and the range of instances over which the carve claims relevance. This frame makes “necessary,” “universal,” and “complete” claims interpretable; it does not fence in candidate generation. Upward abstraction, lateral reframing, or evidence from use may revise any part of the frame, and several incompatible frames or carves may coexist for comparison. The architecture records what each carve claims so that later routing and outcomes can test it. The carve is generative as well as organizational: changing it changes which dimensions receive dedicated reasoning and which mechanisms can be composed, potentially making previously unconsidered solution families locally generable. That possibility is a hypothesis to test, not a consequence of passing the four tests.

2.2 Mandatory Abstraction Before Decomposition

The four tests answer what a concept is made of after the relevant concept has been located. They do not by themselves answer where a concrete stated problem belongs in a persistent graph. A complete construction pass therefore begins with the most specific reusable capability that frames the problem and follows conceptually adjacent *specialization* relations upward until it reaches RootSolver. At every proposed attachment, the model searches existing and plausible siblings and records the parent genus, the child’s differentia, how the higher frame changes the later carve, whether the contracts fit, and whether an informative intermediate parent has been omitted. A broad umbrella

is not enough: an intermediate parent is warranted when it changes routing, exposes a reusable invariant, or changes the dimensions visible below.

The resulting ascent is then traversed in exact reverse order. After that traversal reaches the specific composite, the case applies the four-test procedure to ask what the capability is made of and binds the accepted dimensions through *composition* edges. The relation types must not be conflated: specialization says that a child is a kind or realization of its parent and supplies an abstraction route; composition says that a child supplies a differentiated contribution required to realize its parent’s outward capability. Specialization remains open-world—sibling search does not claim to enumerate every possible kind—whereas completeness is claimed only for the constitutive composition set. Higher abstraction is mandatory for locating a case; additional downward depth remains governed by the marginal-value rule.

```
LOCATE_AND_DECOMPOSE(problem, graph, depth_budget):
  subject <- identify_specific_capability(problem)
  ascent <- [subject]
  while last(ascent) != RootSolver:
    parent <- search_adjacent_parent(last(ascent), graph)
    siblings <- search_observed_and_plausible_siblings(parent, graph)
    if an informative intermediate changes routing or the later carve:
      parent <- insert_or_reuse_intermediate(parent, siblings)
    record genus, differentia, reframing effect, contract fit,
      omitted-parent search, and sibling search
    ascent.append(parent)
  route <- reverse(ascent)
  traverse(route)
  axes <- DECOMPOSE_CONCEPT(subject, depth_budget)
  synthesis <- explain_how(axes, realize=subject.outward_capability)
  return route, axes, synthesis
```

Figure 1: The complete construction motion. The ascent locates the problem in an abstraction hierarchy; the reverse traversal applies that frame before the downward four-test carve. The required record makes the proposed topology inspectable without claiming access to an oracle’s private reasoning.

This procedure constrains the public topology artifact, not the model’s hidden sequence of thought. The model remains responsible for semantic judgment; deterministic validation can check rootedness, edge types, exact route reversal, and artifact integrity, but not whether a parent is insightful or true.

2.3 The Four-Test Search Operator

When decomposing a concept, the operator does not ask “what are the steps to accomplish this?” (that produces a procedural carve). It asks: *what differentiated dimensions must be addressed for this capability to function across the instances it currently claims?* The aim is a candidate low-coupling basis of the capability, not a proof of metaphysically final categories. Four questions generate and criticize candidate dimensions:

Necessity. Relative to the outward capability currently being claimed, if you removed this sub-concept, would the parent still function? If removal causes the promise to collapse, the sub-concept is load-bearing. An economy without selection lacks a mechanism for determining what gets produced. Education without assessment lacks a systematic way to determine whether learning occurred.

Independence. Can you change or reimplement one sub-concept without forcing the same revision in the others? You can redesign an economy’s selection mechanism (from market to planned production) without changing your theory of fairness. You can redesign an educational assessment system without altering the content curriculum. Zero interaction is not required; the practical criterion is lower coupling, visible through counterfactual change propagation, routing overlap, and recurrent joint failures. If two candidates cannot vary or fail separately, merge them, move shared context to their parent, or search for a better cut.

Universality. Does every instance in the range currently claimed for the parent capability contain this sub-concept? Candidate economic axes should recur across hunter-gatherer, feudal, capitalist, communist, and post-industrial systems; candidate educational axes should recur across Montessori schools, doctoral programs, and historical apprenticeships. A sub-concept limited to particular implementations, such as stock-market regulation or multiple-choice testing, is not universal over that range—but it may remain a valid specialization under a narrower claim.

Completeness. For the current outward promise and analytic purpose, if you addressed all the sub-concepts, would you have a functioning instance of the parent capability? If something is missing, there is an axis, interaction, or framing assumption the carve has not represented.

The procedure combines two related but distinct disciplines. Necessity, independence, and completeness echo the Mutually Exclusive, Collectively Exhaustive principle formalized in Minto’s Pyramid Principle [24] and earlier decomposition methodology. Upward classification follows the Aristotelian genus-and-differentia form: name the more general kind shared with siblings, then state what distinguishes the selected child [25]. Downward decomposition uses all four tests; universality has a mathematical analogue in Formal Concept Analysis [26], where a formal concept’s intent is the attributes shared by all objects in its extent. Genus and differentia do not make an observed sibling set complete: specialization is open-world, whereas completeness belongs to a claimed set of constitutive children. The contribution here is to package these disciplines as an LLM-executable construction procedure. Natural human concepts often exhibit graded, prototype-like structure rather than strict necessary-and-sufficient membership [27]; the tests are disciplines for assessing candidate reusable Solver boundaries, not a descriptive theory of human categorization, an oracle, or a proof of a uniquely correct ontology.

Then recurse: each sub-concept is itself a concept. Apply the same four tests. The recursion terminates when a sub-concept is concrete enough to be addressed directly by a single Solver—the point at which the marginal-value rule (Section 3.2) indicates that further decomposition would cost more than it yields.

Four implementation notes are critical. First, the four tests are evaluated by an LLM, which makes them probabilistic judgments rather than mathematical proofs. Human decomposition likewise involves judgment about whether a dimension is truly independent or merely appears so. The algorithm formalizes the structure of that judgment rather than certifying its outcome. Gate rejections and Pathway Memory provide evidence for later revision.

Second, low coupling is the operational aim; real concepts leak into each other. Distribution and Selection in an economy *do* interact. The algorithm seeks a useful cut, not perfect orthogonality. Candidate carves can be compared through change propagation, routing ambiguity, and downstream failure correlation rather than accepted on verbal confidence alone. When leakage causes downstream failures, the Frame Error mechanism (Section 5) propagates upward, signaling that the decomposition itself must be reframed. The leaky seam discussion in Section 3.1 addresses this directly.

Third, the routing sanity check screens candidate decompositions for functional routing failures. Before a decomposition crystallizes as a Sema pattern (a content-addressed specification, detailed in Section 4.5), three concrete domain-specific tasks are generated and routed through the proposed sub-concepts. If a task cannot find a clean home, that may indicate a missing dimension. If a task belongs equally to two dimensions, that may indicate that the axes are not independent enough. This

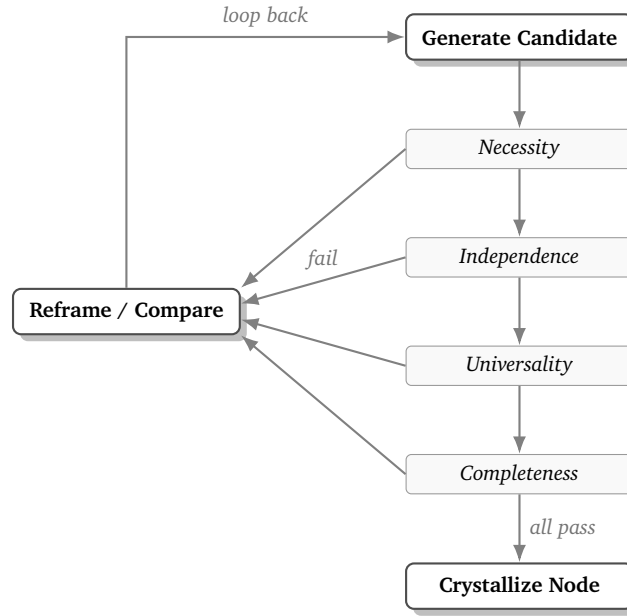


Figure 2: Serving-path control flow for the downward four-test operator. A failed or uncertain test triggers reframing or comparison and another candidate-generation pass; the carve may also remain a labeled frontier alternative. Figure 3 gives one bounded crystallization policy.

empirical check can reveal decompositions that are logically elegant but functionally weak before they enter persistent use.

Fourth, the four tests are an aid to intelligence, not a ceiling on it. A candidate that fails or unsettles a test can remain in the frontier with its failed verdict and rationale, be tried under a different claim frame, or be compared experimentally with the current serving carve. Other decomposition operators may propose structures the four-test search did not reach. Routing, gate, and outcome evidence may later promote such an alternative—or reveal that the test itself encoded the wrong distinction. Crystallization therefore records the best-supported current hypothesis without equating procedural conformance with conceptual truth.

In a deployment of the proposed Solver architecture, a RootSolver first locates a concrete problem through the upward route above; the specific composite or another parent Solver then applies the four tests when decomposing its own concept. Large language models can generate candidate dimensions, evaluate the four tests, and propose reframings; an implementation could therefore search many candidate decompositions and compare their observed coupling and outcomes, subject to the compute and evaluation limits developed below. The model remains responsible for the creative act of proposing a better frame; the operator makes that proposal inspectable and revisable.

2.4 Example: Decomposing the Economy

“Solve the economy” is the kind of task that seems unapproachably vague. Task decomposition produces project-management artifacts: gather data, consult experts, draft policy, implement, evaluate. Conceptual decomposition asks: what are the differentiated, relatively low-coupling dimensions that every economy in the intended range must address?

- **Selection** (what gets produced?): resource allocation, innovation incentive, scarcity resolution.
- **Distribution** (who receives the output?): fairness (procedural, distributive, intergenerational), access, transfer efficiency.

```

DECOMPOSE_CONCEPT(concept, depth_budget):
    candidates = generate_candidate_dimensions(concept)
    axes = []
    for each candidate:
        verdicts = assess_necessity, independence, universality
        if any verdict fails or remains uncertain:
            reframe, narrow the claim, or retain as a labeled frontier alternative
            continue
        axes.append(candidate)
    // Completeness: bounded search (three retries)
    retries = 0
    while NOT complete(concept, axes) AND retries < 3:
        new_candidate = search_for_missing_dimension(concept, axes)
        if new_candidate passes all three tests:
            axes.append(new_candidate)
            retries += 1
    // Routing sanity check: empirical grounding
    test_tasks = generate_sample_tasks(concept, n=3)
    for each task in test_tasks:
        if task cannot route cleanly to exactly one axis:
            reframe axes until routing is clean OR accept with warning
    // Recurse where depth budget permits
    for each axis in axes:
        if marginal_value(axis) > theta:
            axis.children = DECOMPOSE_CONCEPT(axis, depth_budget - 1)
    return axes

```

Figure 3: One bounded crystallization policy for the four-test operator. The completeness search is limited to three retries, while failed or uncertain candidates may remain labeled frontier alternatives rather than disappearing. A routing sanity check screens the candidate carve for missing or overlapping dimensions. Recursion depth is governed by the marginal-value rule.

- **Valuation** (how is worth determined?): price formation, externality accounting, temporal discounting.
- **Coordination** (how do actors synchronize?): information flow, trust, incentive alignment.
- **Stability** (how does the system handle shocks?): shock absorption, feedback regulation, resilience.

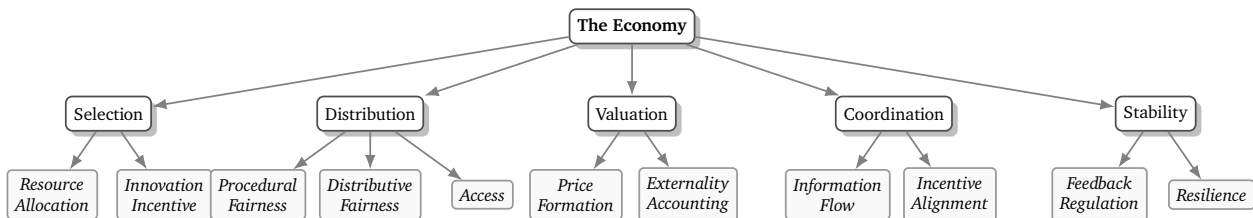


Figure 4: Proposed conceptual decomposition of the economy. White nodes are candidate axes; gray nodes are candidate sub-concepts that recurse further.

This proposed curve treats the five axes as load-bearing, jointly covering the concept and analytically distinct although coupled in practice. It permits selection and distribution, and valuation and coordination, to be examined separately. Comparative decompositions can test whether those separations are useful or whether some axes should instead be merged.

The tree omits implementation actions such as raising interest rates, adjusting taxes, creating jobs, or building infrastructure. Those tasks emerge inside the conceptual structure: “raise interest rates” belongs within Stability → Feedback Regulation, while “adjust taxes” belongs within Distribution → Fairness → Distributive Fairness. The actions are economy-specific; the proposed conceptual axes are intended to recur across economic systems.

Now recurse one level. Distribution → Fairness decomposes into procedural fairness (is the process just?), distributive fairness (is the outcome just?), and intergenerational fairness (is it just across time?). Under this proposed curve, the dimensions are intended to satisfy necessity, independence, and universality. A `ProceduralFairnessSolver` defined by process justice could be called by economic system design, legal systems, hiring processes, hospital resource allocation, and school admissions. Its proposed transfer value comes from the recurrence of process-justice questions across those settings; whether a common Solver actually improves them remains to be measured.

2.5 Example: Decomposing Education

- **Content** (what is worth knowing?): knowledge selection, sequencing, relevance.
- **Transmission** (how does knowledge transfer?): representation, interaction, scaffolding.
- **Assessment** (how do you know learning occurred?): measurement, validity, feedback timing.
- **Motivation** (why does the learner engage?): intrinsic drive, extrinsic structure, meaning.
- **Adaptation** (how does instruction adjust?): diagnosis, differentiation, pacing.

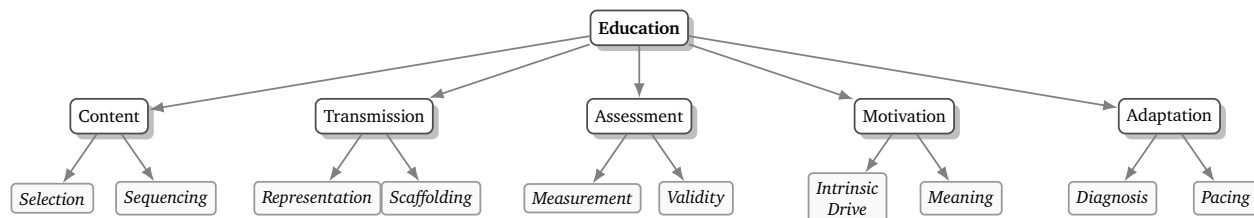


Figure 5: Proposed conceptual decomposition of education, with candidate reusable sub-concepts at depth two.

The proposed axes can vary across forms of education, while several sub-concepts suggest cross-domain correspondences. Assessment → Validity shares the question of whether a measure captures its target with medical diagnostics and software testing. Adaptation → Diagnosis may share a pattern of locating the relevant state with medical triage and debugging.

2.6 Example: Decomposing Game Design

- **Agency** (what can the player do?): action space, consequence, expression.
- **Challenge** (what resists the player?): difficulty calibration, uncertainty, constraint.
- **Feedback** (how does the player know their state?): immediacy, clarity, richness.
- **Progression** (how does the experience change?): mastery curve, revelation, pacing.
- **Coherence** (why does it hold together?): internal consistency, thematic unity, aesthetic integrity.

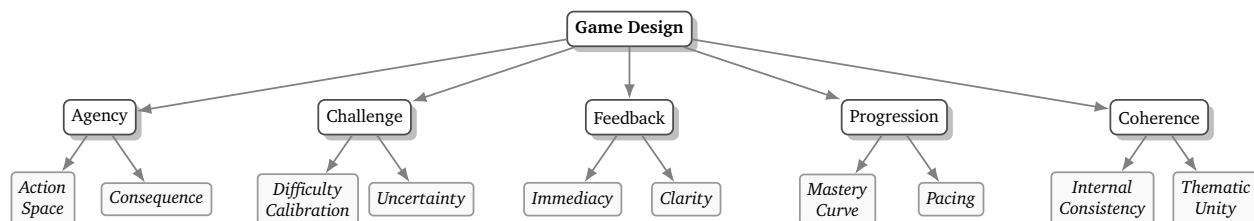


Figure 6: Proposed conceptual decomposition of game design into five dimensions.

The proposed dimensions recur across chess, poker, tag, and digital games, although their prominence varies. Even low-challenge or sandbox forms contain agency, constraints, and feedback in attenuated forms. The sub-concepts may transfer as well: difficulty calibration shares structure with instructional pacing and progressive overload.

2.7 Decomposing Intelligence Itself

The operator should apply to its own target. What are the candidate differentiated dimensions of *intelligence*?

No single decomposition is assumed canonical. Intelligence can be decomposed *modally* (changing frames, working within them, bridging between them), *structurally* (building a model, projecting it forward, evaluating outcomes, updating representations), *functionally* (perceiving, acting, communicating), or in other ways. Different cuts may each be complete relative to a stated analytic purpose.

This paper’s deepest claim is that intelligence should be modeled not only as a structure but as a process capable of restructuring its own representations in response to evidence. Fixed decompositions can support competence; adaptive restructuring is the additional capability of interest here. It allows a system to reshape the categories it uses for thinking when a problem exposes their limits.

Modular plant growth offers a biological analogy for topology-changing adaptation. Plants adjust to heterogeneous environments through distributed flows of resources and information, changing the persistent structure from which later growth proceeds [28]. Their form carries consequences of earlier responses and helps condition later ones. The analogy illustrates the difference between choosing actions within a comparatively stable organization and changing the organization through which future conditions are encountered; it is not evidence for the proposed machine architecture.

The proposed architecture instantiates this second mode through co-evolving topology (§8), rejection-driven reframing (§4.1), ascent from depth (§3.4), and convergence across paths (§3.3). The adjective *fractal* refers to the pattern’s recurrence across scales: a structure may compose into a larger one while remaining open to decomposition, retention, reuse, and revision.

Concrete decompositions remain useful as tools: modal cuts can assign roles, structural cuts can audit cognitive capacity, and functional cuts can reveal interface gaps. The adaptive-restructuring criterion within fractal intelligence is the capacity to adopt, revise, and discard such cuts as evidence and circumstances change.

2.8 Cross-Domain Convergence: Why Reasoning Highways Form

Across these three domains, several candidate structural correspondences appear:

Feedback. Assessment in education, feedback in game design, and feedback regulation inside economic stability each contain a narrower operation: using signals to estimate a system's state. A shared Solver would be justified only if constituent matching and probe routing showed that this operation is genuinely common.

Engagement. Incentive alignment in economics and motivation in education may share operations involving intrinsic drivers, extrinsic structures, and meaning, while remaining distinct parent concepts.

Adjustment. Adaptation in education and coordination in economics may both include sensing state, adjusting a path, and managing tempo.

Evaluation. Validity in educational assessment, internal consistency in game design, and procedural fairness in economic distribution all compare system behavior against a governing criterion, but they remain distinct evaluative concepts.

The four tests explain the intended recurrence. By selecting for necessity, low coupling, universality over a declared range, and completeness relative to an outward capability, they favor abstract dimensions that are candidates for reuse beyond the problem that produced them. If those candidates capture real invariants, the uniform contract gives structurally similar problems a common destination regardless of origin domain. Whether they recur because they are found structure rather than attractive labels requires independent Root-only repetitions plus matched ablations that remove, scramble, or replace the upward-first and four-test operator.

Convergence creates the possibility of compounding, but two claims must be separated. *Within-concept* compounding is a Solver improving through repeated instances of its own concept; the mandatory-abstraction construction records cross-problem and cross-domain structural reuse but not improvement, because it has no independent execution, Pathway Memory, acceptance gates, or outcomes (Section 11). *Cross-concept* compounding, such as transferring what was learned about uncertainty in weather to finance, is the stronger and least certain extension. The empirical program is to map which concepts transfer, including negative transfer, rather than assume universal compounding.

2.9 When Decomposition Goes Wrong

Two archetypal failures deserve attention. A *leaky seam* looks clean on paper but implicit assumptions bleed across the boundary. If a SelectionSolver's acceptance criteria embed a distributive objective, the distributive assumption has crossed the seam. The remedy is to strengthen the interface so the assumption must be explicitly typed. A *misaligned cut* separates variables that must co-vary; the remedy is to revise or merge the boundary so the coupled variables are handled together.

The cost of a bad cut can be severe. If a parent solver makes a misaligned cut at the root, descendants may inherit a flawed ontology and optimize the wrong variables. The Frame Error mechanism (Section 5) therefore allows leaf-node friction to challenge a parent decomposition. The mandatory-abstraction construction demonstrates higher-parent insertion and multi-parent topology, but its deterministic structural acceptance is not a feedback-triggered Frame Error or semantic repair (Section 9.2).

2.10 The Stability of Solved Structure

Two kinds of stability must not be conflated. A solved structure should remain addressable: its decomposition, execution route, outputs, gate results, provenance, and observed outcome should not evaporate when the task ends. But persistence does not make its decomposition canonical. The same problem may admit several useful carves, and later evidence may expose a seam that the incumbent missed. Stability belongs first to identity, retention, and learned reuse—not to one immutable ontology.

The architecture therefore separates three layers. An *inclusive archive* retains evaluated solution graphs, including failed routes when they provide negative knowledge. A *selective serving layer* promotes recurrent, validated, or niche-leading nodes and subgraphs into active infrastructure; Pathway Memory learns when to route through each. An *experimental frontier* receives a bounded budget for fresh, convention-resistant carves when similarity is low, routing confidence is poor, frame errors recur, stakes are high, or an exploration schedule fires. Cheap carve sketches can be screened before full execution. This is a portfolio in memory: nothing is epistemically discarded, but familiar problems normally exploit one evidence-backed incumbent rather than execute every retained carve.

Evaluated structures act like maintained roads: tasks generate traffic through them, recurrent routes can become highways, and failed routes can be bypassed or rebuilt. Reuse can amortize decomposition without making its cost disappear. Expected task cost is $C_{\text{route}} + C_{\text{solve}} + p_{\text{explore}}C_{\text{carve}}$: familiar tasks make the exploration probability small; novel or failing tasks make it larger. Whether retained structure lowers total cost enough to justify the archive, routing, and occasional re-carving is an empirical question. Matched-compute evaluation must charge every exploration call rather than declare the overhead negligible by construction.

2.11 Evolutionary Encapsulation and Emergence in Reverse

For the purposes of this paper, complexity is observer-relative: it is the gap between a system’s full internal detail and what an observer can meaningfully describe or hold in mind for the purpose at hand. An exact atomic-scale description of an ordinary macroscopic system is generally too detailed to track, even though compact laws capture many of its regularities. We make such systems cognitively workable through higher-level units such as cells, organs, organisms, and machines. Encapsulation does not remove internal complexity. It gives the system a bounded exterior through which it can participate in organization at another scale.

Natural intelligence is evolutionary in origin. Across major transitions in individuality, entities capable of independent replication became integrated into higher-level individuals. Cooperation, communication, division of labor, mutual dependence, and sufficiently low internal conflict allowed the collective to function as a new unit [3, 4]. This makes nested encapsulation an organizational precedent worth examining, not evidence that the Solver architecture works. Simon’s stable-intermediate argument explains why such layering can make further complexification more tractable [11]. Levin’s multiscale-competency framework motivates the stronger cognitive hypothesis: cells, tissues, organs, and organisms can exhibit goal-directed competence at nested scales, with a collective presenting as one agent-like unit to the level above [29].

Seen through this precedent, the Solver architecture can be read as an engineered reversal of boundary assignment. Biological evolution supplies no a priori global contract: in some lineages, lower-level entities became integrated and a higher-level individual emerged. The architecture assigns the maximal boundary first: the `RootSolver` encapsulates the complete problem space of its deployment, and each submitted problem is selectively decomposed within it. Every child may contain a further Solver tree, yet a leaf and a compatible completed subtree can present the same contract boundary. Recursive closure therefore preserves encapsulation as an interface property.

The proposed interface guarantees the form of the boundary, not that the boundary is conceptually correct, that its implementation is independent, or that the work behind it is good.

Contract-bounded Solvers serve as artificial cell-like units: a model, human, program, or team provides the cognitive interior; the contract provides the membrane, and the Manifest its compressed public description. A composite parent translates its obligation into child Tasks, inherited constraints, and acceptance criteria, then routes, synthesizes, and evaluates what returns. Its synthesis obligation is the local membrane contract: it states how the exact active child contributions, in their parent-local roles, plus routing, boundary management, integration, and evaluation realize the single outward capability promised above. A child may recur beneath many parents, remaining addressable while each assigns a different Task and role. Because a composite can itself serve as a child, the same whole-part relation nests recursively. Each composite Solver is thus both a whole relative to its children and a bounded component relative to its parent—what this paper calls *constrained nested emergence*: the parent specifies conditions for coherence without prescribing every internal operation. Declared synthesis is inspectable evidence of a composition design, not proof that useful operational emergence occurred. Feedback may also update routing.

Conventional emergence begins with local entities and rules and yields a higher-order pattern that was not placed directly—flocking from flocks, markets from traders, perhaps consciousness from neurons. The surprise is at the top: define the micro, and the macro may astonish. The proposed architecture reverses the direction of boundary assignment. It names the parent-level capability first, while leaving open which child configurations, cross-branch reuse, and routes will eventually sustain it: define the macro, and the micro may astonish. This inversion is why the paper uses the phrase *emergence in reverse*.

That the contract is designed does not by itself disqualify this deliberately weak, operational sense of emergence. A cellular automaton's rules are written, but a glider need not be placed; what matters is whether the higher-order pattern arises from lawful interactions rather than being individually scripted. The analogous claim here is not that the system lacks constraints, but that no designer must specify in advance which compatible configurations persist, which nodes attract enough traffic to become reasoning highways, or how deep each branch will go.

The macro boundary is therefore assigned top-down, while useful routes, candidate higher parents, nodes that densify into highways, and the depth reached by each branch may emerge through use. There is no fixed architectural depth because any Solver may decompose again while remaining one unit to its parent. Every actual execution is nevertheless finite because it runs under a finite budget; the marginal-value rule decides whether further decomposition is worth spending that budget. The mandatory-abstraction construction does not yet establish this phenomenon: it derives a theory-conditioned scaffold and records declared composition rather than independent child action. It shows how the pattern can be represented, not that operational emergence occurred.

At the interface level, this can be read as selective decompression: the marginal-value rule decides where a bounded unit should expose additional internal reasoning. Retention and promotion perform the complementary abstraction by encapsulating recurrent working structure as a new addressable unit. The resulting design rule is to encapsulate a recurring structure when its public obligations are stable enough to name and test, while keeping the unit revisable. A system that never encapsulates risks being overwhelmed by its own interactions; a system that encapsulates too early risks freezing a carve that evidence has not yet earned. When recurrence makes generality earn its name, the structure becomes a candidate for sealing behind a hash (§4.5).

3 A Theory of Depth

Section 2 defines how to locate a problem upward and decompose it downward; this section governs how deep to go. At every node, the choice between direct solution and further decomposition, repeated across levels [13, 10], produces a *solver tree* in which the same contract applies to workers and orchestrators. The *Universal Solver Tree* is the aggregate of all such solver trees across agents in the system. Its universality concerns the intended breadth of problem coverage, not a uniquely correct ontology. Within each constituent tree, the root boundary has maximal scope and minimal declared specialization relative to its descendants, while each decomposition step proposes a narrower, more specific distinction. This is the depth-wise counterpart of selective decompression: broad public claims at the root, selectively exposed detail below. When outcome evidence validates a distinction and later reuse demonstrates transferable value, retention can preserve it as accumulated structural intelligence. Depth alone is structural elaboration, not evidence of intelligence. The aggregate grows through vertical refinement by decomposition and lateral discovery by reframing, with the marginal-value rule deciding when additional depth is expected to pay.

3.1 Selective Depth

Under realistic cost models, returns to depth are diminishing while costs accumulate. Moreover, uncertainty is unevenly distributed. Uniform depth wastes budget on easy spans and starves the hard ones. The remedy is selective depth: concentrate refinement on sub-concepts with high uncertainty, brittle dependencies, or large impact. A *hard seam* is a boundary whose failure could collapse the whole plan. Refining or repairing such a seam can prevent erroneous work from propagating through the subtree below it.

A solver tree provides explicit, asymmetric compute allocation: it can route inference budgets toward hard seams, resolve simpler spans shallowly, and retain an inspectable graph of work. The additional scaling axis proposed here is *composition* across typed boundaries and trust domains, where invocation context can be separated and a detected failure can return a structured request for reframing.

Fractal intelligence can be instantiated by one model simulating a tree or by many specialized models (Section 4.3). The relevant comparison is with systems that decompose implicitly: the wager is that making decomposition explicit, typed, and persistent enables reuse, localized learning, and cross-organization sharing.

The history of sports analytics illustrates this principle concretely. For decades, baseball evaluated players at a fixed granularity: batting average, runs batted in, earned run average. The Oakland Athletics under Billy Beane decomposed “player value” one level deeper—on-base percentage, slugging percentage, plate discipline—and discovered that conventional metrics were masking variables associated with wins. The next generation went deeper still: player-tracking systems decomposed movement into sprint speed, route efficiency, launch angle, spin rate, and defensive range. In the vocabulary of this paper, each variable becomes a candidate sub-concept with its own Solver, acceptance gate, and Pathway Memory. Baseball’s unusually decomposable domain and rapid scoreboard limit generalization from the example. The marginal-value rule likewise assumes bounded productive depth: θ marks the point beyond which further decomposition ceases to pay. The wager is that conventional practice sometimes stops above that floor; Beane found leverage by analyzing player value at a finer conceptual resolution.

3.2 The Marginal-Value Rule

The economic governor for depth is a ratio test whose lineage runs through information value theory [30], ecological foraging [31], and rational metareasoning [32, 33]: deepen if and only if the expected improvement divided by the incremental cost exceeds a domain-appropriate threshold θ . Formally, at any node n :

$$\frac{\mathbb{E}[V(n_{\text{deep}})] - V(n_{\text{shallow}}) + U(n)}{C(n_{\text{deep}}) - C(n_{\text{shallow}})} > \theta. \quad (1)$$

Incremental cost includes the *interface load* of an added boundary—translation, serialization, routing, synthesis, monitoring, and semantic loss—while the value term must credit any error or risk reduction that makes such friction worthwhile.

De Sabbata et al. [34] applied this directly to LLM reasoning depth. This paper extends the principle from a single model’s reasoning depth to a tree of contract-bounded solvers, governing not just how long to think but where to grow the hierarchy. The bonus term $U(n)$ is an exploration credit inversely proportional to the density of Pathway Memory at node n , in the spirit of the upper-confidence-bound rule of Auer, Cesa-Bianchi, and Fischer [35]: nodes the system has rarely visited receive a temporary subsidy that converts the rule from pure exploitation into an exploration–exploitation trade-off, with the cold-start probe phase of Section 11.9 as its practical implementation.

In practice, an orchestrator implements this through a control loop:

```
ALLOCATE-DEPTH(branches, budget, theta):
  while budget > 0:
    ratios <- empty map
    costs <- empty map
    for each branch b in branches:
      (dV, dC) <- b.Consult("estimate_next_step")
      ratios[b] <- dV / dC
      costs[b] <- dC
    b_star <- argmax(ratios)
    if ratios[b_star] < theta:
      break
    b_star.Execute()
    budget <- budget - costs[b_star]
  return collect_results(branches)
```

Figure 7: The marginal-value rule as a control loop. The Consult surface makes the economic governor computable; the exploration credit $U(n)$ is omitted from the loop for clarity.

The Consult surface grounds its estimates in Pathway Memory rather than zero-shot self-evaluation (Section 5.1). Each Solver maintains structured logs of past executions, recording which problem classes it has seen, what depths yielded improvement, and at what cost. A query like `estimate_next_step` returns a lookup: “For problems structurally similar to this one, deepening with pattern Y has historically yielded a 12% quality improvement at cost \$0.05.” Where historical data is unavailable, the estimate defaults to a calibrated prior; when neither is available, the system allocates a fixed exploration budget to shallow probes before committing depth.

3.3 Topology Freedom

The contract constrains what crosses boundaries (typed artifacts and acceptance gates) but not how Solvers are arranged or sequenced. A task-oriented pipeline typically passes an *instruction* (“do this specific thing”), which tends toward a fixed, sequential topology. A conceptual Solver passes an expression of *intent* (“resolve this concept”). Because intent admits multiple fulfillment strategies, a parent need not prescribe the child’s execution path; topology can instead adapt to observed evidence. Two persistent relation types must nevertheless remain distinct. A *specialization* edge says that the child is a kind or realization of the parent and supplies an abstraction route. A *composition* edge says that the child provides a differentiated contribution to the parent’s outward capability and may become executable work. Node identity remains the capability contract; relationship meaning and parent-local role belong to the edge. The same contract supports radically different execution topologies:

Sequential chains: output from Solver A to Solver B to Solver C. *Evidence-gated routing*: a parent examines a child’s result and decides where to send it based on what the result contains—not based on its position in a sequence. *Fan-out and fan-in*: parallel solvers pursuing competing hypotheses, synthesized at a typed boundary. *Cycles*: a creative Solver and a verification Solver cycle repeatedly until the acceptance gate clears or the marginal-value rule halts. *Flat orchestration*: a parent routes among peers based on evidence, no tree depth. *Recursive nesting*: any of the above inside a single Solver that presents a single typed interface.

The term “Universal Solver Tree” names decomposition’s rooted origin, not the final topology. One unchanged Solver may serve several parents, each supplying a typed Task, constraints, acceptance criteria, and edge-local role; reuse shares the capability, not the parent concepts or invocation contexts. Fan-in and multi-parent reuse therefore produce a directed graph, with $|E| > |V| - 1$. The current reviewed concept-topology profile requires a rooted acyclic graph, even though bounded execution protocols may support cycles.

At each level, a parent’s routing, synthesis, and evaluation are part of the intelligence under test. Manifest matching proposes compatible Solvers—one implementation could use vector similarity over declared Manifests—while Pathway Memory ranks them from prior evidence. Semantic similarity is a routing heuristic, not proof of contract compatibility. The conceptual structure is the blueprint, while each problem induces a route through it.

Topology freedom extends to the *decomposition path* itself: for a given problem, the tree is not unique. Consider designing a city. One decomposition cuts by *process* and elevates “flow” to a top-level sub-concept, with movement-in-homes appearing deep inside the flow branch. Another cuts by *spatial object* and elevates “buildings” and “streets” at the top, with movement appearing twice in the tree—once inside buildings, once inside streets—without ever being decomposed as a first-order axis. Both decompositions may satisfy the four tests; neither is canonical. At the depth where movement appears, they may converge on the same MovementSolver: by importing its hash where the concept is adopted, or by semantic matching where it was independently derived. Content-addressing guarantees identity of adoption, not collision of derivation; that is the synonymy limit of Section 11. The hypothesis is therefore *path-indifferent but concept-convergent*: it does not require one canonical tree, but predicts that useful atoms can recur across trees. Path-indifferent here means admissibility, not equal outcomes; the serving layer may learn that one carve helps more. The archive may retain both city graphs when they expose complementary structure.

3.4 Jumps and Reframing

When vertical decomposition bottlenecks—for example, when the marginal-value rule detects a flat improvement trajectory—more computation inside the same invariants may have little value. A lateral jump instead changes the frame so that a different region of solution space becomes locally available.

Alternative interpretations can themselves form a Discovery population (Section 6.4). A `WhyChainSolver` searches beneath the stated problem, and a `ConstraintInversionSolver` tests inherited assumptions. Orthogonal Insight is one long-range variance operator available to that protocol: an unrelated seed is elaborated into a coherent causal world by a target-blind World Builder; a fresh Solver receives that world and the target; and a separate bridge compiles the operative relation into a real candidate mechanism [1]. Orthogonal Insight calls the in-world step a *Blind Solve*, meaning purpose blindness: the Solver receives the local target but not the experimental purpose or subsequent bridge task. The world changes what is locally ordinary before the system asks whether the resulting mechanism is useful.

Candidate reframings may then enter the companion ontology controller. Current occupancy and curator judgments can guide further commissions, while a later `ReduceSolver` could select candidates for downstream decomposability or present their trade-offs for user judgment. Automatic saturation detection and the reduction stage remain proposed controls.

The meta-perspective is maintained through parallel solvers at different levels of abstraction. Parent nodes see how the parts relate; children see how the parts work. Critically, this is bidirectional: a child solver may discover something that reshapes the parent’s understanding—the Feedback surface is the channel through which children inform parents. A Frame Error at a child propagates upward not as a failure but as evidence that the parent’s decomposition needs revision.

Mandatory classification ascent (§2.2) is distinct from topology-changing ascent. Every case must locate itself on a specialization route; a new higher parent is added only when sibling comparison, recurrence, or a Frame Error supports a revised topology. This evidence-driven revision of the categories through which work is routed is *ontological accommodation*. Reframing is thus not only lateral; it can also be *ascending*. Topology-changing ascent has two independent triggers, only one of which is failure. The first is the Frame Error above: a child fails, and the failure signals that the parent’s decomposition was wrong. The second is *recurrence recognition*: a child succeeds, but while solving its sub-problem it recognizes that the conceptual structure it is handling is not specific to this branch at all—the same structure recurs across unrelated problems. Nothing has gone wrong; something general has been found. When either trigger fires, the execution path does not merely descend the current branch; it *teleports* to the newly recognized abstraction, where the pattern becomes a general concept with its own Solver. Subsequent problems encountering similar structure at any depth can now route directly to this elevated Solver, bypassing the decomposition that first discovered it. This is how reasoning highways (§5.2) emerge at runtime: depth finds the pattern—through failure or through recurrence—ascend generalizes it, and future problems teleport to the generalization rather than re-deriving it. The trajectory through the tree is therefore not a monotonic walk but a graph traversal with occasional vertical jumps; depth and abstraction are coupled, not orthogonal.

4 The Solver Contract

The Solver Contract specifies how concept-defined abilities compose across boundaries. The distinction between Solver and agent is architectural: an agent is an implementation (a model, a program, a human), while a Solver is the interface that implementation presents to the system. The contract defines a five-surface namespace that may be implemented by a lightweight script, a frontier LLM, or a swarm of thousands. Manifest and Execute form the required core; Consult, Verify, and Feedback are discoverable extensions declared by the Manifest. “Five-surface” names the common interface vocabulary, not a claim that every lightweight Solver implements all five.

A Solver specializes through three complementary mechanisms: *weight adaptation* (fine-tuning for a narrow problem class), *tool composition* (invoking external capabilities), and *execution harness* (a deterministic protocol guiding a general model through a domain-specific process). The contract is indifferent to which mechanism operates behind the interface. Section 4.4 develops this opacity in detail: a single agent may implement many Solver contracts, a single Solver may orchestrate many agents internally, and the lifecycle from general-purpose hat to specialized system is governed by the same economic rule that governs depth.

The contract’s intellectual ancestry is precise. Hoare [36] established verification through preconditions and postconditions. Meyer [37] operationalized Design by Contract. Liskov and Wing [38] extended behavioral subtyping. Findler and Felleisen [39] showed that contracts compose for higher-order functions—the recursive pattern Solvers require. The proposed extension here is that the boundary is not merely a validator but can return a typed request for upstream restructuring when the gate rejects. When that request changes the working carve rather than merely triggering another attempt, the boundary participates in ontological accommodation (Section 3.4). The five surfaces defined by the Solver Contract are shown in Table 2.

Table 2: The five defined surfaces of the Solver Contract. Manifest and Execute form the required core; the other surfaces are discoverable extensions.

Surface	Core Question	Purpose
Manifest	What can you do?	Declares the concept this Solver reasons about, its operations, input/output types, and any extended surfaces. Enables discovery and operation-level compatibility checks.
Execute	Can you do the work?	Takes a typed Task, returns a typed Result. The surface that performs conceptual work.
Consult	How would you approach this?	Open consultation: cost/quality estimation, clarification, rationale requests, context-sharing. Enables the marginal-value rule.
Verify	Why should I trust you?	Post-execution assurance: test vectors, reproduction recipes, verifiable evidence.
Feedback	How did you do?	Receives structured evaluation from downstream, or signals a Frame Error requiring a lateral jump. Enables localized learning.

A *core-compliant* Solver implements Manifest and Execute. An *extended* Solver exposes any additional surfaces listed in its Manifest; a *full-surface* Solver exposes all five. Compatibility is evaluated for the operation actually requested rather than inferred from a shared name.

4.1 Gates That Redirect

At each boundary between Solvers, three things happen: (1) mode is declared (the Solver explicitly states its cognitive stance); (2) output is typed (the artifact crossing the boundary has explicit structure); (3) acceptance is gated (an `AcceptSpec` defines what must be true for the artifact to cross). The strictness of the gate scales with the seam. At soft seams (low-stakes internal handoffs), acceptance may be a lightweight shape check. At hard seams—structurally critical junctures or boundaries crossing trust domains—acceptance is non-compensatory: if any critical criterion fails, the artifact is rejected regardless of other scores.

These create *seams that bite* [40]: boundaries tight enough to maintain distinct cognitive modes, yet flexible enough to preserve internal freedom. Each gate acts as the endpoint of the subtree beneath it, judging that subtree’s final artifact. The intended benefit of isolation is to reduce compromise among cognitive modes that would otherwise share one context.

Mode blending remains useful during divergent phases, where soft seams preserve permeable boundaries. Hard seams become relevant during convergence, when candidates must survive adversarial scrutiny. The `AcceptSpec`’s strictness is therefore a function of the cognitive phase.

Deterministic gates evaluate mechanically decidable properties: schema conformance, hash matches, and unit tests. They avoid language-model evaluator error for those checks, but incomplete criteria and implementation defects can still produce false assurance. *Probabilistic gates* rely on LLM evaluation. These inherit evaluator vulnerabilities: sycophancy, laziness, and *evaluator collusion*—where generators and gate evaluators from the same model family share blind spots. A structural mitigation is to use adversarial evaluators at hard seams, potentially from different model families, ensemble juries, or dedicated red-team solvers. The proposed Discovery Protocol (Section 6.4) therefore assigns generation and curation to distinct cognitive roles; whether model-family separation further improves reliability remains an empirical question.

A third outcome complements pass and reject: *override-with-documentation*. When an artifact fails quantitative criteria but the failure reflects the limits of the criteria rather than the limits of the work, the gate permits passage with a typed justification citing what the criteria missed, why it matters, and how to test the override. This prevents deadlocking on problems where quantifiable criteria cannot capture everything. The Ethical Reasoning Protocol (Section 6.6) instantiates this mechanism.

The Solver Contract specifies a *floor*, not a ceiling. The acceptance gate verifies the typed artifact (the minimum that must be true) but does not limit what else flows across. A solver can pass its full reasoning tree, rejected alternatives, and confidence distributions alongside the verified output—by reference rather than as raw text, avoiding context-window bloat—and the downstream solver can use whatever helps.

4.2 The Transaction Model

The fundamental interaction is a transaction. A Solver receives a `Task` object and returns a `Result` object:

Table 3: The Task–Result transaction.

Task Fields	operation, inputs (typed), acceptance_criteria (AcceptSpec), budget (time/cost/compute limits)		
Result Fields	outputs (typed artifact), status (success/partial/fail), stop_reason (completed/budget/quality)		

This forces handoffs to be structurally explicit. A Task payload crossing a hard seam:

```
{
  "operation": "sema:Optimize#0921",
  "inputs": {
    "mechanism": "Solid-State Battery Electrolyte",
    "target_metric": "ionic_conductivity"
  },
  "acceptance_criteria": {
    "schema": "sema:AcceptSpec#e4f1",
    "criteria": [
      { "metric": "conductivity_mS_cm", "comparator": ">=", "threshold": 1.0 },
      { "metric": "thermodynamics", "status": "pass" }
    ]
  },
  "budget": { "max_compute_usd": 0.50 }
}
```

Figure 8: Example Task payload crossing a hard seam, with typed inputs, acceptance criteria referencing a Sema hash, and budget.

The schema field references a content-addressed Sema specification (sema:AcceptSpec#e4f1), allowing a receiver to verify that both parties reference the same immutable criteria. The hash establishes identity and integrity, not that the criteria are correct or sufficient. The status: partial option enables the anytime property: a Solver that exhausts its budget returns the best result so far.

Figure 9 shows the full lifecycle of a transaction across a hard seam.

4.3 Recursive Composition

Any Solver may decompose its Task into sub-Tasks and delegate to child Solvers, applying the marginal-value rule at each level (Figure 10). Constraints are inherited fully and monotonically: if the root Task specifies “no network access,” every descendant inherits that constraint.

The Solver Contract is recursively closed in a structural sense: a composite subtree can present the same boundary as a leaf when the requested operation appears in its Manifest, the Task satisfies its declared input and constraint requirements, every descendant inherits constraints without weakening them, and the composite synthesis returns the declared Result type for evaluation against the caller’s referenced AcceptSpec. These are operation-level conformance conditions rather than identity by label. They preserve interface-level encapsulation as the tree deepens but guarantee neither semantic equivalence, correctness, nor faithful execution; those remain the work of decomposition, typed specifications, gates, and outcome evaluation.

The apex case is the RootSolver: a Solver whose Task is an open-ended goal and whose cognitive operation is triage—determining what kind of problem this is and routing it. The RootSolver’s Pathway Memory is the architecture’s most consequential site of compounding: it is the only node that sees how all problems enter the system.

The contract is invariant across five levels of ceremony: (1) natural language coordination, (2) pattern-enriched dialogue using Sema patterns, (3) lightweight Solvers with partial surfaces, (4) rigorous Solvers with full contracts and non-compensatory gates, (5) recursive orchestration with deep trees. Companion experiments suggest the largest gains may be front-loaded: the biggest reported improvement occurs between unstructured and pattern-enriched coordination [41]. The five surfaces provide a common interface for three coordination modes: *hierarchical* (parent calls

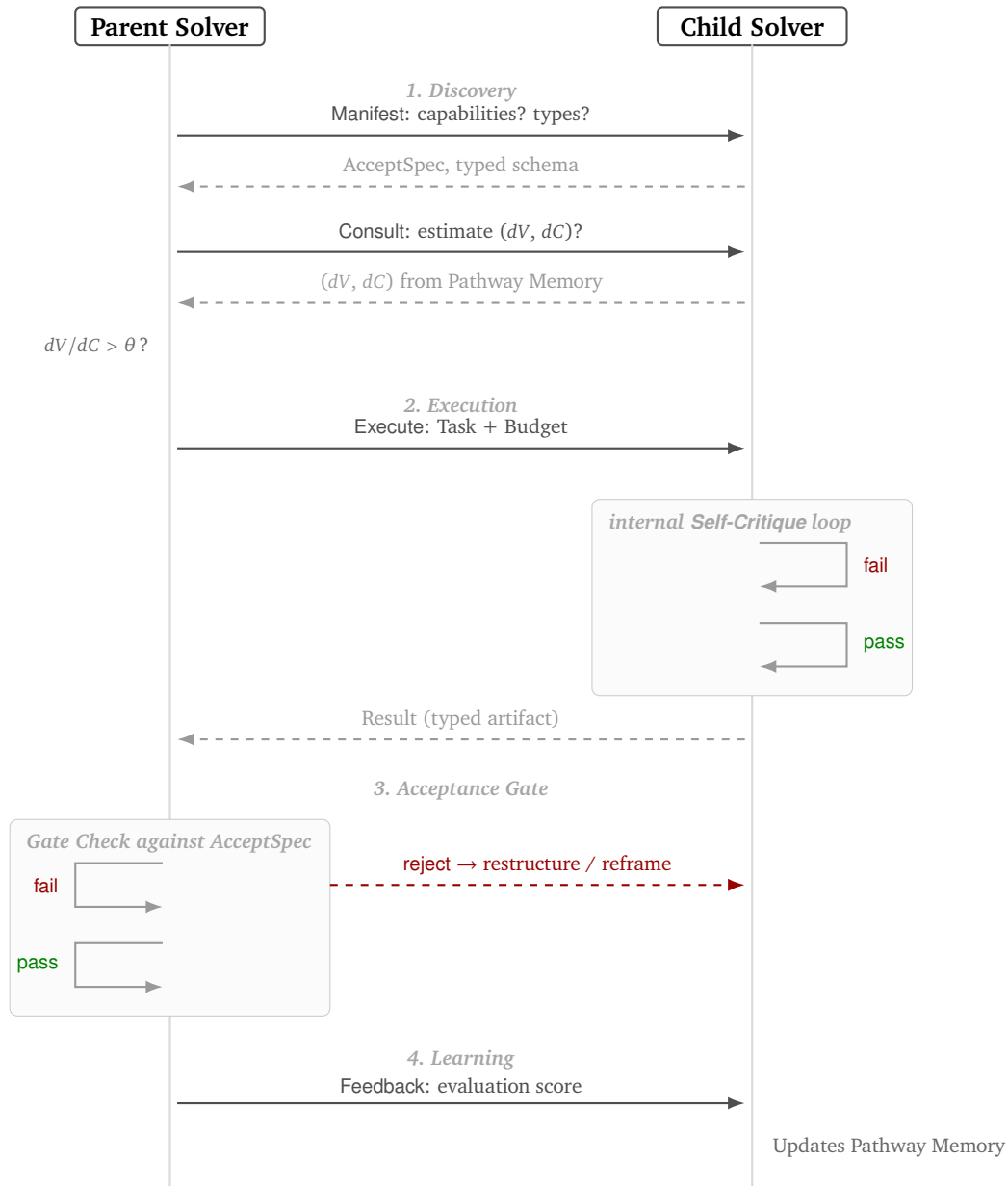


Figure 9: Solver transaction lifecycle across a hard seam. Phase 3 is the boundary-level acceptance gate—the “seam that bites.” Rejection routes to restructure, lateral reframe, or graceful degradation (status: partial).

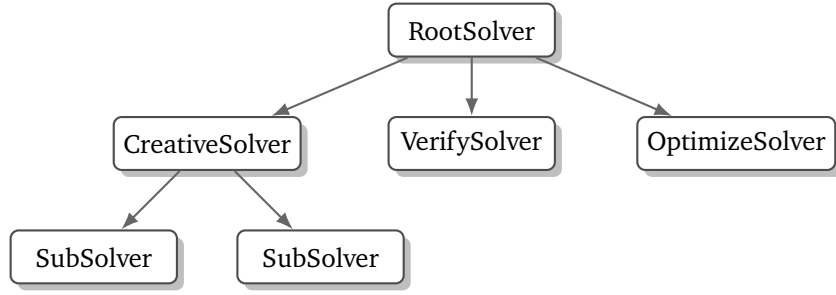


Figure 10: Recursive decomposition. Each Solver may delegate to children, applying the same contract at every level. Depth is governed by the marginal-value rule.

child), *peer-to-peer* (Solvers with compatible extended endpoints engage directly), and *stigmergic* (Solvers interact through a shared medium [6] rather than direct calls).

4.4 What Sits Behind a Solver

The five-surface contract specifies what crosses the boundary, not what sits behind it. Whether a Solver is implemented by three models and a knowledge graph or by a bash script, the caller sees the same typed interface and judges it against the same `AcceptSpec`. This architectural opacity is deliberate and admits three deployment configurations:

- *One agent, many Solvers.* A single LLM instance registers multiple Manifests and activates the appropriate concept, bounds, and `AcceptSpec` based on the incoming Task. The solver tree is a conceptual structure, not a deployment topology—hundreds of conceptual nodes may be served by a handful of agents, each wearing many hats. This mirrors human organizations where a single expert fulfills multiple roles depending on context.
- *One Solver, many agents.* An `IncentiveAlignmentSolver` might internally route to a frontier model for novel mechanism design, a small fine-tuned classifier for known pattern recognition, and a Python script for game-theoretic equilibrium calculation. The caller sees one Solver presenting one Manifest. Behind it is a team. This is itself a form of conceptual decomposition, but it is invisible to the caller and governed by the Solver’s own internal logic rather than the parent’s routing decisions.
- *One Solver, one agent.* A dedicated fine-tuned specialist. The simple case.

A useful but limited property of the *one agent, many Solvers* configuration is that the contract creates an explicit change of role. If generation and evaluation run in fresh contexts under different Manifests and `AcceptSpecs`, the model is asked to reconsider an artifact against declared criteria rather than continue the same generative trajectory. This may mitigate some same-trace self-correction failures, but shared weights and priors remain, so role separation does not guarantee independent judgment. It nevertheless provides a bootstrapping path: a frontier model can initially serve several concept-defined interfaces, while the resulting graph and evaluations remain addressable and individual nodes may later be replaced by specialized harnesses or models. An internal harness can also log and expose its steps; the contract’s distinction is that exposed steps become typed, independently routable boundaries rather than remaining implementation details.

4.4.1 The Solver Lifecycle

This structural opacity admits a natural lifecycle. A Solver begins as a “hat” on a general-purpose LLM—no specialization, just the contract. As invocations accumulate and Pathway Memory reveals that a concept recurs across many domains, investment may become justified. The marginal-value rule compares the expected improvement from specialization with its cost, using invocation frequency and cross-domain breadth as evidence. A handful of uses may justify only a lightweight harness; frequent cross-domain use may justify fine-tuning or a dedicated model.

The specialization sequence typically unfolds structurally: an execution harness is added, then a fine-tuned classifier for routine cases, then domain-specific tools, and finally a full internal team. Recent work on Meta-Harness [42] provides empirical evidence that this harness layer is load-bearing: automated optimization of the scaffolding around a single model produces accuracy improvements of 7–16 points, and this optimized harness transfers across different model architectures. In the architecture proposed here, Meta-Harness optimizes what sits behind one node; the contract governs how those nodes compose. The architecture produces the data that makes the decision to specialize legible.

4.4.2 From Generalist Role to Specialized Solver

A Solver may begin as a general model operating within a narrow semantic neighborhood. Its specificity comes from execution harnesses, tools, weight adaptation, and its position in the graph: a `StabilityRegulationSolver` receives a typed subproblem isolated by upstream decomposition and emits an artifact filtered by an `AcceptSpec`. Repeated use may then justify progressively stronger specialization under the marginal-value rule. A prompted role can therefore develop into a specialized system when its invocation history supports the investment.

4.5 Sema: Content-Addressed Concepts

Collective thinking requires a shared repertoire of reusable cognitive strategies—patterns that prescribe *how* to think about a class of problems, not merely what a concept denotes. Sema [41] provides this by content-addressing pattern definitions: the cryptographic hash of a specification becomes its identifier, so two Solvers referencing the same hash are guaranteed to reference the same immutable specification—the same principle by which Git content-addresses code, applied to cognitive operations rather than source files. The hash guarantees specification-byte identity only; interpretation, execution, and semantic discovery require additional mechanisms. Content identity needs no central registry, while discovering a compatible specification still depends on an unresolved semantic matching procedure; neither the construction artifact nor the reference implementation validates semantic equivalence. *Semantic identity*—whether differently encoded specifications express the same operative invariants—is therefore an unresolved equivalence claim, not a property of the hash. The proposed architecture’s operational components are formalized as Sema patterns (Table 4), making the protocol self-describing.

Not every cognitive artifact should become a Sema pattern. A hash can in principle be minted over anything—a specific chain of reasoning, a single day’s deliberation, an idiosyncratic problem decomposition—but like words, cognitive patterns gain *infrastructural value through generality*. A pattern that fires once has the cost of a Sema entry without the benefit; a pattern invoked across many domains compounds in value with each use. This mirrors the Universality criterion of the decomposition algorithm (§2): good sub-concepts, and good Sema patterns, are the ones that recur. The complement holds on the other side: the Understanding Graph [6] stores specific instances of thinking—narrative traces of reasoning toward a particular understanding. When a recurring structure is recognized in such traces, it may be promoted into a Sema pattern. The promotion

is the same ascent mechanism described in §3.4, but across memory systems: specific-thinking (Understanding Graph) matures into reusable-strategy (Sema) when recurrence makes generality earn its name.

Table 4: Mapping from architectural components to content-addressed Sema patterns.

Concept	Sema Pattern	Hash	Role
<i>The Solver Contract (Five Surfaces)</i>			
Manifest	Card	#f1e1	Capability advertisement
Execute	CognitiveSolver	#c9f9	Universal solver atom
Consult	SocraticLoop	#4c98	Pre-execution consultation
Verify	Validate	#c745	Schema verification
Feedback	PerformanceSignal	#db20	Downstream evaluation
<i>Proposed Architectural Mechanisms</i>			
Good Cuts	Decompose	#7ee9	Conceptual splitting
Selective Depth	RecursionDive	#9db4	Vertical tree traversal
Marginal-Value Rule	MarginalValueRule	#ee80	Economic stop-condition
<i>Infrastructure</i>			
Task	Task	#d9f9	Atomic unit of work
Result	Solution	#084c	Typed output artifact
AcceptSpec	AcceptSpec	#e4f1	Typed acceptance boundaries
Solver Tree	UniversalSolverTree	#0715	Collective knowledge graph

Sema is a library of over 450 specialized thinking strategies, including *SocraticLoop* (clarification before generation), *SteelmanCheck* (mandatory counter-argument), and *PreMortem* (failure simulation). Each is a content-addressed executable specification: not a description but a contract specifying invariants, preconditions, failure modes, and typed dependencies—a strategy that can be invoked, not merely cited. When a Solver hydrates a Sema pattern, it loads scaffolding: a step-by-step protocol constraining its cognitive path, with invariants the runtime can verify at each step. Content-addressing prevents unnoticed mutation of the referenced specification: any byte-level change produces a different hash, although execution can still diverge. Refinement therefore lives *behind* the hash, not inside it: the specification is immutable while the Pathway Memory and harnesses behind it may improve (the skeleton–muscle split of Section 7); a change to the specification itself is a new version, and version migration at scale remains open.

Content-addressing provides three properties useful for collective thinking. *Specification identity*: matching hashes imply matching canonical specification bytes, not merely matching function signatures. *Granular negotiation*: if fields are independently addressed in a Merkle tree, Solvers can agree on one referenced component while disputing another. *Immutable reference with mutable organization*: hashed fields are immutable under a given identifier, while unhashed metadata can be reorganized without changing that identifier. These properties establish stable reference, not shared interpretation or correct execution.

4.6 Productive Extremes

Structural separation may protect cognitive modes that interfere when co-activated. Like a cell membrane, a typed boundary permits unlike processes to coexist while preserving their distinct operating conditions. Dual-process research [7, 43, 44] reports such interference in human cognition: analytical verbalization can disrupt insight problem-solving [45], deliberate thought can suppress divergent generation [46], and dual-task interference degrades performance even in simple tasks [47]. Related effects appear in language models: task-switching within conversational history can degrade

performance [48], generalist multimodal models may underperform specialists under objective interference [49], and outputs can regress toward the mean in creative tasks [50]. A concurrent analysis spanning 152 findings across agent prompt architectures catalogs 21 labeled interference patterns and argues that “the agent that resolves the conflict cannot be the agent that detects it” [51]. Together, these findings motivate the hypothesis that interference can become a structural limitation and that typed conceptual boundaries may mitigate it; they do not establish that such boundaries are the only remedy.

The defining test is whether boundaries between concepts produce results unavailable to an individual model under matched prompting, sampling, memory, and compute. If they do, the boundary is not only a validator but an *intelligence contributor*: its separation and typed exchange change the reachable result. Released Orthogonal Insight artifacts illustrate target-blind world construction, purpose-blind in-world solving, and extraction, while the companion Ontology study documents curator-driven redirection [1, 2]. Existing artifacts illustrate these ingredients but leave the defining comparison open because they lack a matched monolithic baseline and independent coding of mechanism novelty. A machine system could maintain dozens of competing hypotheses or alternative causal worlds in parallel, beyond an individual’s working memory, while the uniform contract makes their artifacts composable. This is an architectural capacity, not a result established by the current studies.

4.6.1 Illustrative Trace: Orthogonal Insight

To see the proposed mechanism in concrete form, consider a released Orthogonal Insight trace, later analyzed in the companion Ontology study [1, 2]. The problem: design a retirement system for variable-income workers. Three separately prompted phases use asymmetric information boundaries.

Phase 1: World-Building. A World Builder, given the seed word “theatrical,” constructs an alien physics with no knowledge of the downstream problem. In the resulting world, causality follows narrative logic: nothing becomes real until it is witnessed, events resolve at the moment of maximum dramatic impact, and the fundamental force is not gravity but *Tension*, the pressure of unresolved dramatic potential.

Phase 2: Blind In-World Solving. The in-world Solver is purpose-blind: it receives the retirement problem and generated world, while the experimental rationale and later extraction task are withheld. Because the runner forwards the generated world verbatim, the seed word may remain visible; the information barrier concerns experimental purpose rather than seed identity. Working as though theatrical physics were real, the Solver designs a Tension Ledger tracking “dramatic weight” rather than dollars, Contribution Ceremonies where sincerity matters more than amount, and Timing Guilds that read narrative readiness rather than account balances. Coin and pooled funds still exist in the fictional world, but balance no longer governs retirement; dramatic weight, sincerity, and narrative readiness do.

Phase 3: Extraction. A third Solver translates back to reality. “Tension Ledger tracking dramatic weight” becomes a hardship-adjusted sacrifice score: contributing 5% of income when income has dropped 40% scores higher than contributing 10% at stable income. The resulting design, Witnessed Sacrifice Retirement, inverts standard retirement logic entirely, measuring what it cost you to save rather than how much you saved.

The trace makes the search hypothesis concrete: world construction changes the premises before in-world solving, and a fresh extractor compiles the resulting relation back to reality. It illustrates the hypothesis rather than testing causal superiority; the study lacks a compute-matched monolithic baseline and does not isolate phase separation.

The companion study also implements population-level curation. Its execution harness assigns proposal generation to fresh Explorers and confines the recurring Taxonomist—modeled here as a `TaxonomistSolver`—to classification, admission, redirection, and restructuring of a persistent typed graph. The graph database functions as an external cognitive prosthetic: no uninterrupted Taxonomist context must remember the whole population, yet each session can inspect and revise its represented structure. When the Taxonomist rejects with a structural diagnosis and commissions a mechanism-changing retry, this paper calls the operation *structural coaching*; when a candidate does not fit the current categories and the graph itself changes, it is ontological accommodation. One trace moves a retry from detection-to-trigger toward ownership routing. Another adds *Reframe Inconsistency* after judging a candidate outside seven existing top-level stances. These records establish executed redirection and category revision, not their causal effect on creativity or recovery of a natural ontology.

4.6.2 Isolation vs. Self-Similarity

Purpose separation can be implemented through ordinary sequential API calls and careful system prompts. The contract contributes a uniform structure for composition and retention: evaluated trees can persist as solution graphs, recurrent structures can be promoted into content-addressed patterns, and the Consult surface, typed gate outcomes, and Pathway Memory make depth decisions, local evaluation, and later routing first-class architectural objects (Sections 4.5 and 3.2). Isolation supplies the cognitive mechanism; self-similarity and persistence make it recursively composable and reusable.

4.6.3 Structural Reliability and Safety

This architecture aims to move some safety burden from generation weights to explicit boundaries [8]. It targets one possible *alignment tax*: capability lost when generation and safety evaluation must compromise inside the same cognitive process. Where a gate can evaluate the relevant hazard, generation and verification need not be optimized in the same context. This may reduce one source of capability–safety compromise, but it does not make safety a property of the boundary alone; the result remains limited by the gate’s coverage, evaluator quality, and the behavior of the underlying Solvers (Section 11).

A strict gate introduces a mechanical risk: deadlock in a rejection spiral. Repeated rejection may indicate a flawed decomposition, an unattainable or badly specified criterion, or evaluator error. After N materially similar rejections, the marginal-value rule can halt leaf retry and request diagnosis at the parent boundary: revise the carve, revise the criterion with documented authority, or stop (Section 3.4). If no justified repair clears the gate, the anytime property activates: status: partial with the best intermediate result available.

Where Solvers have isolated parameter sets, fine-tuning one does not alter another’s parameters. This prevents updates in one Solver from causing catastrophic interference in unrelated Solvers, though it does not prevent forgetting within an individual Solver [52]. The creative Solver need not be trained directly against the verifier’s objective, but pretrained knowledge or supplied context may still let it anticipate the criteria; parameter separation reduces cross-objective interference rather than guaranteeing cognitive purity. The unit of training shifts: instead of one large generalist, one may train a portfolio of specialists whose value comes from how they compose through the contract.

The contract can improve structural reliability by catching some failures before they propagate. Under an illustrative independence assumption, a ten-step chain of 90%-reliable Solvers yields 35% end-to-end success ($0.9^{10} \approx 0.35$). Acceptance gates can stop detected failures, but end-to-end reliability remains limited by both Solver error and gate coverage, including false accepts and false rejects. A typed boundary can similarly contain a prompt injection only when the compromised output

violates a criterion the gate can verify; a malicious but well-typed artifact may still pass. The boundary is therefore a partial firebreak, not a security guarantee. A companion demonstration supports the narrower coordination claim: in a multi-agent trading swarm, natural-language coordination produced catastrophic loops with fat-tail latency (up to 51 turns and 317 seconds), while the reported protocol condition had median 15.4 turns and 203 seconds with no observed fat tail [41]. The comparison does not isolate which protocol component caused the difference.

5 Localized Learning

The proposed architecture supports learning at four levels. First, each node can accumulate Pathway Memory—performance data from its invocations—to ground later marginal-value estimates and routing decisions. Second, a node may sharpen at its specific job through fine-tuning, tool integration, harness refinement, or better estimates from experience. Third, observed divergence or redundancy can motivate splitting, merging, or reparenting nodes. Fourth, routing models can learn which paths work for which problem types, co-evolving with the specialists they route to. Conceptual identities remain addressable while the topology and the capabilities behind them remain open to revision.

5.1 Partition by Subproblem Type

When an acceptance gate detects a failure at a seam, the responsible Solver boundary becomes an explicit unit for diagnosis and localized feedback.

When evidence suggests that a Child Solver is failing because the Parent made a leaky cut rather than because of the child’s own limitations, the Feedback surface can return a *Frame Error*. This requests parent-level diagnosis and may trigger the lateral jump of Section 3.4 instead of another leaf retry.

The diagnostic question is when to declare a Frame Error rather than retry the leaf. The proposed Feedback policy treats two patterns as *insolubility signatures*. (A) *Convergent failure under structural diversity*: when several sub-solvers built on materially different internal mechanisms (different harnesses, parameter families, or tool sets) all violate the same AcceptSpec clause, repeated agreement becomes evidence of an upstream ontological leak, although leaf noise and gate error remain possible. (B) *Oscillatory constraint contradiction*: when outputs satisfying clause A repeatedly break clause B, and vice versa, the parent may have posed an impossible problem. Either pattern can request a lateral jump (Section 3.4) after the specified evidence threshold. To prevent indefinite reframing, both signatures are governed by an *ontological cooling* schedule: as compute budget drains or the reframing taxonomy of Section 3.4 saturates, the trigger threshold rises monotonically, so the system eventually falls back to the override-with-documentation mechanism (Section 4.1) rather than looping inside the meta-loop.

Localized learning rests on a distinction the contract makes structurally explicit: *protocol transfer* versus *weight transfer*. Protocol transfer distributes one immutable protocol identity—the decomposition graph, public acceptance clauses, and gate structure identified by a Sema hash—but byte-identical content does not establish that the protocol will transfer successfully across domains. Weight transfer, or local weight adaptation, changes parameters and can instead be scoped to a single Solver’s execution boundary: LoRA-style fine-tuning [53] need not update unrelated parameter sets. The typed Task is intended to act as a semantic bottleneck, with an upstream node translating a domain-specific payload (currency reserves, affect state) into the abstract variables the specialist expects and a downstream node translating its output back into domain language. Whether this bottleneck adequately limits domain-vocabulary contamination is empirical. This realizes the function–arguments analogy of Section 11.6: the hash fixes the function specification; the arguments vary by invocation.

Alongside the specialization mechanisms of §4.4, *Pathway Memory* records explicit performance data (quality, depth, cost) across problem classes and grounds the marginal-value estimates. It may use a typed reasoning-capture graph [6] or a leaner statistical index optimized for routing speed. The standardized Feedback surface supports three timescales: *immediate* rejection data, *medium-term* pathway attribution, and *long-term* real-world outcomes.

5.2 Reasoning Highways and Credit Assignment

The credit assignment problem—determining which node in a deep hierarchy caused a downstream failure—is typically the hardest obstacle to learning in hierarchical systems. The Solver architecture addresses *local* credit assignment through statistical accumulation at common nodes; *global* routing credit assignment remains an open challenge (Section 11).

When structurally diverse problems route through the same conceptual decomposition nodes, each node can accumulate a frequentist signal without explicit weight training. These convergent paths—*reasoning highways*—would form when the routing layer sends structurally similar conceptual sub-problems to the same specialists across origin domains. A `DistributionSolver` handling tax-policy allocation, network bandwidth routing, and scholarship allocation would generate localized performance records through use: the same node is observed across different inputs and outcomes. These are comparable observations rather than controlled experiments.

Reasoning highways share *Sema scaffolding*—the conceptual decomposition, acceptance gates, and sub-solver types—while domain-specific payloads may be handled by separate node instances. The routing model learns the structural correspondence among those instances: tax-policy allocation and scholarship allocation may decompose into the same conceptual sub-problems without overloading one node with both domains.

The reuse cycle requires problems to route through genuinely common conceptual structures; the contract makes reuse composable and outcomes comparable, but does not guarantee independent convergence (Section 11). In a decentralized commons, the backward learning flow also needs protection from fabricated penalties. A *Receptivity Gate* requires a rejecting orchestrator to submit a typed `FailureTrace` naming the violated `AcceptSpec` clause; the upstream Solver verifies the trace before accepting it, and invalid or hallucinated feedback is dropped.

6 Generalized Decomposition Protocols

This section presents ten generalized protocols: nine author-designed decompositions of recurring cognitive needs and one General Problem-Solving Protocol derived by applying the four-test algorithm to problem-solving itself (Section 6.9).

Each protocol is intended as a cross-domain decomposition of a recurring cognitive need. Its specification abstracts away the domain payload, although its chosen boundaries and examples may still encode assumptions inherited from the author or model. Each extends chain-of-thought reasoning across contract-bounded Solver invocations, whether implemented by one model or many: Chain-of-Thought [54] made reasoning linear, Tree-of-Thoughts [55] made it branching, and these protocols make it boundary-separated and persistent. Domain knowledge enters through the Solvers and contexts invoked within a protocol. Because the protocols attach to recurring concepts rather than one prescribed application domain, they are candidates for the architecture’s highest reuse tier rather than proven domain-agnostic structures.

The nine specialized protocols are design proposals rather than outputs mechanically generated by the algorithm. The formalization in Section 2 attempts to systematize the intuitive process that produced them.

Several protocols below contain an inherent causal order: prediction precedes scoring, and interpretation precedes planning. Their decomposition nevertheless concerns continuous, co-constraining faculties rather than chronological tasks. The temporal sequence follows data dependency; matched tests must determine whether separating the faculties reduces interference (Section 4.6).

Ten universal needs, ten protocols:

- **Producing with quality:** Collaborative Writing (Section 6.1)—“Does this meet the standard?”
- **Understanding people:** The Human Emulator (Section 6.2)—“How should I respond to this person?”
- **Evaluating:** PURE (Section 6.3)—“Is this worth pursuing?”
- **Discovering:** The Discovery Protocol (Section 6.4)—“What are the best solutions from a diverse field?”
- **Predicting:** Temporal Ensemble Forecasting (Section 6.5)—“What will happen?”
- **Choosing rightly:** Ethical Reasoning (Section 6.6)—“What is the right thing to do?”
- **Verifying:** The Truthseeking Protocol (Section 6.7)—“Is this true?”
- **Making:** The Creation Protocol (Section 6.8)—“How should this be brought into existence?”
- **Solving:** The General Problem-Solving Protocol (Section 6.9)—“What is any problem made of?”
- **Self-regulating:** Meta Protocols (Section 6.10)—“Is the system still the right shape?”

6.1 Collaborative Writing: Decomposing the Concept of Textual Quality

The protocol decomposes the concept of *quality-constrained production*—what it means to produce something to a standard—into five intended low-coupling dimensions: *substance* (what material must the artifact contain?), *structure* (how does it organize?), *generation* (produce freely, with evaluation elsewhere), *evaluation* (does it meet the standard, across independent quality dimensions?), and *refinement* (surface quality on a structurally complete artifact).

Writing makes the decomposition vivid because most of its cognitive work is invisible. A single sentence in a scientific paper activates an entire evaluative apparatus in any trained reader [5], and the writer performs these evaluations silently, in parallel, each one bleeding attention from the others. The imperative to be precise suppresses vividness. The awareness of audience dampens technical depth. Conventional task delegation reproduces this: the writer still self-censors during generation while the editor catches style, logic, and structure in a single read.

The protocol separates structural production from concurrent monitoring. In the production path, a `ScopeDiscoverySolver` identifies every distinct idea the text must teach—ideas rather than sentences—and produces a flat list. A `SequencingCoherenceSolver` orders them by dependency. A `UnitConstraintSolver` writes one paragraph per idea, while a `StructuralVerificationSolver` checks the argument scaffold for unsupported claims, orphaned examples, and mechanisms disconnected from the claims they are meant to explain. A `VoiceRefinementSolver` adds connective tissue to the structurally complete document. The marginal-value rule governs further decomposition, from a short memo that stops at one paragraph per idea to a high-stakes argument that spawns sub-solvers for units needing deeper treatment.

Concurrent monitors externalize otherwise implicit evaluation. Multiple `ObserverSolvers`—a `RepetitionSolver`, a `ContradictionSolver`, an `IntentSolver`, and an `AudienceReceptionSolver`—receive the evolving artifact and return typed annotations describing location, severity, and the cognitive mode that triggered the signal. They annotate rather than rewrite, separating evaluation from revision. No extension to the Solver Contract is required: an `ObserverSolver` is a standard `Solver` whose `Task` is the current artifact state and whose `Result` is a set of typed annotations.

A `TextCategorizationSolver` assigns each paragraph a conceptual type—definition, mechanism, example, property, motivation, claim—and flags structural violations such as several consecutive paragraphs performing the same rhetorical function. This operates on the concepts underneath the text rather than on surface wording alone.

The proposed observers illustrate all three specialization mechanisms. A `RepetitionSolver` equipped with a sentence-level embedding index can perform pairwise cosine similarity across every paragraph: *tool use*, moving a suitable comparison outside the language model. A `StructuralCoherenceSolver` can walk a deterministic checklist (does every forward reference resolve? is every concept introduced actually used?): an *execution harness*. A `ContradictionSolver` could be fine-tuned for greater sensitivity than a generalist prompted to “check for contradictions”: *weight adaptation*. The contract allows each observer to use the mechanism appropriate to its task; any performance advantage would require measurement.

Even a single observer can decompose further. A `RepetitionSolver` might spawn a `LexicalEchoSolver` (surface reuse) and an `IntentionalRepetitionSolver` (deliberate rhetorical callback). The detector can be optimized for sensitivity while the intent solver separately judges what to preserve.

Structural process and concurrent monitoring are designed to be independently composable. The same pattern could apply to music composition, with harmonic and rhythmic monitors, or to interface design: the persistent object would be quality-constrained production rather than writing alone.

6.2 The Human Emulator: Decomposing the Concept of Empathetic Understanding

The Human Emulator illustrates *variable depth*. It decomposes “How should I respond to this person?” into situation, emotion, intent, response, and boundary. A `SituationSolver` reconstructs history and relational dynamics. An `EmotionSolver` produces a structured model of what the person likely feels and why. An `IntentSolver` infers the need beneath the surface request, a `CalibrationSolver` adjusts tone and register, and a `BoundarySolver` enforces ethical constraints as a hard gate.

These faculties are not a relay race but co-variant state monitors: an emotional spike changes the probability distribution over latent intents; a resolved intent reframes the emotional reading; a boundary violation rewrites the response register. Typed artifacts represent these facets separately while allowing them to constrain one another continuously rather than only sequentially.

A simple greeting may resolve in one shallow pass. An ambiguous, high-stakes situation can justify deeper recursion: `EmotionSolver` spawns sub-solvers to model conflicting emotional signals, `IntentSolver` distinguishes the stated request from the underlying need, and `BoundarySolver` tightens its gate. When the words say “I’m fine” but the trajectory suggests otherwise, `EmotionSolver` may spawn competing `EmotionalHypothesisSolvers` for masked distress, trust-testing, and genuine reassurance. An acceptance gate then composes rather than prematurely collapses those hypotheses. The contrast illustrates the marginal-value rule: routine interactions remain shallow while uncertain, consequential ones receive more analysis.

The surface-latent gap is the ordinary texture of human communication, not an edge case. Suppose a collaborator answers “whatever” after repeated incremental confirmations and rising brevity. A `LatentIntentSolver` can treat the trajectory as evidence that the operative need is not another clarification but “stop iterating and deliver the complete package”; a `MetaIntentSolver` asks

why that need has become urgent now. A surface-only response might apologize and ask whether to continue, thereby repeating the failure. The example does not establish superior performance, but it makes the proposed decomposition concrete: surface meaning, latent need, and change-over-time are different objects of inference.

Beneath the hypotheses, an `AffectiveStateMachine` tracks emotional trajectory: engaged → guarded → testing trust. `BoundarySolver` spawns its own specialists: a `ManipulationDetector`, a `DependencyGuard` against parasocial attachment, an `EscalationSolver` for determining when to recommend a human professional. The intended result is a compositional system whose depth can scale with the human situation rather than a chatbot defined only by a personality prompt.

6.3 PURE: Decomposing the Concept of Viability

PURE asks whether an idea—a business, research program, or policy—is worth pursuing. In one context window, excitement about the mechanism can soften parsimony, an expansive vision can weaken uniqueness, and feasibility can inflate perceived transferability. The protocol therefore decomposes *viability* into four independent, non-compensatory gates: minimality (*Parsimonious*), independence (*Unique*), feasibility (*Realizable*), and breadth (*Expansive*). Under its traffic-light rule, exploration proceeds only if no gate is Red; each gate declares its concept, evaluates evidence through adversarial steeldom, and emits a typed verdict.

Each gate is itself a decomposition point. A `ParsimoniousSolver` may compare competing compressions, a `RealizableSolver` may invoke the Orthogonal Insight Protocol (Section 6.4), and an `ExpansiveSolver` may stress-test candidate transfer domains. A five-second screen and a week-long investigation are therefore the same protocol at different depths. The same treatment applies to frameworks such as OODA, SMART, SWOT, and PESTLE: treat each dimension as a Solver and govern its depth economically.

PURE is also a building block within other protocols. Whenever any protocol needs to decide “should we continue down this path?”—whether a `CreationSolver` evaluating a candidate framing, or a `DiscoverySolver` screening early-round outputs—it can invoke PURE as a lightweight viability check. The proposed Discovery schedule below uses this distinction: early rounds would apply a quick `PURECheck`, while later rounds would scale to full PURE with deep decomposition.

6.4 The Discovery Protocol

The Discovery Protocol addresses *population-based discovery*. It decomposes discovery into variance, selection, novelty, composition, and saturation—dimensions that apply wherever a solution space must be searched. Orthogonal Insight is one previously published variance operator within this broader design [1]; a Solver realization can expose it behind an `OrthogonalInsightSolver` boundary. The companion Ontology study implements a distinct map-and-redirect controller for candidate-population state [2]. Diversity-preserving portfolio selection, saturation-based stopping, and progressively realistic validation remain proposed. Within the Solver-based realization developed here, the protocol offers one way to combine lateral expansion with selective depth and persistent reuse.

As proposed here, the protocol organizes these dimensions into two phases behind typed boundaries: “What solutions exist if I think in this specific way?” (generate, asked by many parallel Solvers each bound to a maximally distinct cognitive mode) and “Which of these are genuinely different and worth further evaluation?” (reduce, asked by a Solver whose faculty is evaluation and composition rather than generation).

GeneratorSolvers form the population: each bound to a maximally distinct cognitive mode—different seed constraints, different world-models, different optimization targets. Each is instructed to develop its declared premises before downstream evaluation: a Solver operating under “all actions are reversible” physics reasons within that world rather than optimizing immediately for ordinary feasibility, while one reasoning from “trust is zero” develops the consequences before a later stage judges social acceptability. The separation does not erase the model’s prior knowledge; it aims to keep early feasibility and consensus pressure from collapsing the population before distinct mechanisms have been articulated. The intended diversity comes not from noise but from being *precise in different directions*.

In the full proposed protocol, a ReduceSolver evaluates and composes the population’s outputs at a typed boundary. Reduction is not a single operation but a repertoire of specialized Solvers, each appropriate to a different problem structure. An AggregateSolver performs mathematical combination via proper scoring, ensemble methods, or intersecting invariants across outputs; in forecasting, the distribution of predictions may itself be the answer, its spread revealing systemic uncertainty and its clusters revealing attractor states. A TournamentSolver would conduct adversarial selection through structured debate and comparative judgment. A PortfolioSolver would retain quality-diversity by preserving niche leaders for different regions of the solution space; MAP-Elites [56] and novelty search [57] pursue a related goal over predefined behavioral dimensions with stochastic variation. A SynthesisSolver would merge structurally compatible outputs by intersecting hard constraints and priority-unioning soft ones.

A TaxonomySolver classifies admitted outputs into a growing graph of mechanisms, outcomes, and structural categories [2]. Structurally similar candidates can occupy the same graph region, while candidates judged unrepresented by the curator can create new nodes. The graph then becomes a navigable population map rather than merely a list of winners. A declining rate of node creation could serve as one saturation signal, but its value as a stopping rule remains untested.

The proposed ReduceSolver would route among these modes based on the population’s typed outputs, itself an instance of the topology freedom described in Section 3.3. One proposed design requirement is *alien preservation*: unusual branches receive enough maturation and evidence before familiarity or early feasibility judgments may eliminate them [2]. A non-compensatory novelty criterion is one possible implementation, but it risks protecting difference without utility and was not tested in the companion study. The required balance between structural diversity and downstream evidence therefore remains part of the research program.

A proposed reduction schedule would deepen evaluation as the field narrows. Early candidates would receive cheap criticism or mechanism-equivalence checks; survivors would receive progressively more detailed decomposition, adversarial comparison, simulation, evidence review, expert judgment, and—where safe—real-world testing. The marginal-value rule would govern this allocation: evaluation becomes more expensive only as the value of distinguishing the remaining candidates rises.

The companion study incorporates previously released Orthogonal Insight artifacts—target-blind world construction, purpose-blind in-world solving, and mechanism extraction—and implements only the map-and-redirect portion of this broader design: fresh graph-conditioned Explorers and recurring Taxonomist sessions in three Studio conditions [1, 2]. The Taxonomist qualitatively accepted candidates, rejected with diagnostic redirection, and could restructure a persistent candidate ontology. The study did not implement the ReduceSolver repertoire, tournament selection, hard novelty gates, saturation stopping, or progressive validation described above.

6.5 Temporal Ensemble Forecasting: Decomposing the Concept of Prediction Across Time

Temporal Hindsight Learning [58] is a training methodology that produces calibrated forecasters by distilling reasoning traces from a hindsight-equipped teacher into a temporally blind student. The training process is not itself a thinking protocol, but it produces an artifact that is: a *Temporal Checkpoint Ensemble*, where each checkpoint is a model frozen at a different point in history, and each checkpoint is a Solver. This is the Discovery Protocol applied to forecasting. A population of ForecastSolvers—the 2020 checkpoint, the 2021 checkpoint, the 2022 checkpoint—each reason from a different knowledge horizon. Each sees a different slice of history, carries different priors, and is blind to different futures. The 2020 checkpoint knows nothing of COVID’s economic aftermath; the 2022 checkpoint has internalized it but not the AI investment surge. Their disagreements are not noise but *signal*: when checkpoints that agree on structural drivers suddenly diverge on a specific prediction, the divergence point reveals when an outcome first became structurally predictable. A ReduceSolver composes these temporally diverse forecasts—not by averaging but by preserving the causal reasoning from each horizon and identifying where independent knowledge states converge on the same structural driver.

Each checkpoint further decomposes its forecast through a *Forecasting Pentagon*: five non-substitutable angles (structural, economic, political, base-rate, temporal) that would suppress each other if entangled in a single pass. A solver reasoning about institutional physics (“auditing a factory takes months”) operates in a different register from one reasoning about game-theoretic incentives (“the regulator must act to preserve credibility”). The ensemble is thus doubly diverse: across time horizons and across causal lenses. The ReduceSolver receives a matrix of typed forecasts (five angles from each of n temporal checkpoints) and the acceptance gate requires convergence of *structural drivers* across horizons, not agreement on specific outcomes. When three checkpoints spanning different knowledge states independently identify the same causal mechanism, that mechanism has survived a temporal adversarial test unavailable to any single checkpoint.

Checkpoint history supplies an epistemic distance not obtained merely by varying a prompt: models frozen in 2020 and 2023 reflect different available evidence and training horizons.

6.6 Ethical Reasoning: Decomposing the Concept of Right Action

When prediction and normative evaluation share one undifferentiated generation, each can distort the other: forecasts may be bent toward a preferred conclusion, while moral reasoning may select convenient predictions. This paper calls the resulting failure *smuggled normativity*. The Ethical Reasoning Protocol operationalizes Hume’s is–ought distinction [59] as an architectural boundary: *calculate, then judge*. A PredictionSolver, tasked with description, outputs a typed PredictionLedger: branching future states across multiple time horizons, each with an explicit causal mechanism and a confidence envelope that degrades with distance. The downstream ValuationSolver then evaluates which causal pathways matter, at what confidence, and over what horizon, and emits a typed ScoreSheet before the explicit normative override stage.

The architecture also includes a PrincipleSolver that acts as both an acceptance gate and a formal override. If a top-ranked option trips a sacred value, a catastrophic tail risk, or a procedural precedent, the solver emits a typed JudgmentNote rather than silently adjusting prediction weights. The resulting DecisionRecord preserves two explicit layers: the quantitative base and the qualitative override. This is the contract’s override-with-documentation mechanism (Section 4.1) applied to ethical reasoning. The typed artifacts make the empirical forecast (*what may happen*) and the normative choice (*why an action was selected*) separately inspectable, while not guaranteeing that either layer is free from contamination.

Consider an autonomous system evaluating whether to reroute supply chains away from a politically unstable region. The `PredictionSolver` outputs a `PredictionLedger` with branching scenarios—sanctions probability, port disruption timelines, alternative supplier lead times—each with causal mechanisms and confidence envelopes. The `ValuationSolver` ranks options purely on cost and reliability. But the top-ranked option concentrates economic harm on a vulnerable workforce. The `PrincipleSolver` emits a typed `JudgmentNote` overriding the ranking: the override cites the specific ethical constraint, the specific prediction it overrides, and the cost of the override. The `DecisionRecord` is auditable end-to-end: a reviewer can see exactly what the model predicted, exactly what it chose, and exactly why the two diverged.

The protocol is designed to reduce cross-contamination between prediction and judgment and to make their relationship inspectable. It does not establish moral correctness.

6.7 The Truthseeking Protocol: Decomposing the Concept of Epistemic Warrant

The Truthseeking Protocol introduces *global accretion*: verified factual claims persist as content-addressed records that later checks can reuse.

When a monolithic model evaluates a claim, it may run source evaluation, logical consistency, and cross-referencing in a single entangled pass. A highly credible source’s authority can suppress detection of a logical flaw; strong empirical evidence can dampen scrutiny of unstated assumptions. By isolating *who said it* from *what the logic dictates* from *what the evidence shows* across hard boundaries, the proposed protocol aims to reduce such epistemic interference.

Where other protocols decompose by cognitive faculty or population diversity, the Truthseeking Protocol decomposes by *verification depth*, governed by the marginal-value rule. The layers represent how far to recurse (verification depth), while the epistemic dimensions within Layer 2 (source provenance, logical coherence, evidential correspondence, adversarial contestability) represent the conceptual decomposition of what epistemic warrant is made of. Layer 0 (cache): has this exact claim been verified before? A content-addressed lookup checks the claim’s hash against the verified-claim cache, allowing an eligible cached verdict to return at near-zero marginal cost. Layer 1 (structural coherence): does the claim contradict the verified cache? A `ConsistencySolver` maps the claim against the existing knowledge topology. Layer 2 (epistemic decomposition): a `ClaimExtractorSolver` isolates falsifiable statements, a `ProvenanceSolver` evaluates provenance independent of content, a `CoherenceSolver` checks internal consistency independent of the source, a `CorrespondenceSolver` queries external data, and a `ContestabilitySolver` constructs the strongest counter-case. Each operates behind the contract, isolated from the others’ outputs until they compose at a typed boundary. Layer 3 (cross-domain specialists): high-stakes or novel claims route to domain experts in the Cognitive Commons. Layer 4 (empirical verification): gathering primary data or running simulations.

Consider the claim that “the Bank of Japan has raised interest rates to 3%.” If a verified-claim record already exists from a prior check, Layer 0 resolves instantly. If novel, Layer 1 finds no contradiction with cached monetary policy data. At Layer 2, the `ProvenanceSolver` traces the claim to an anonymous social media post, the `CorrespondenceSolver` finds no corroboration from Reuters or the central bank’s own announcements, and the `ContestabilitySolver` notes that the claimed rate would represent a multi-decade high for the institution. The marginal-value rule halts: three independent epistemic failures converge without incurring Layer 3 costs, and the typed output identifies the exact deficit—unverified source, no corroboration, historical implausibility.

The proposed architectural addition is the *accretion loop*. A claim surviving deep verification is recorded as a content-addressed verified-claim entry: a tamper-evident reference to the claim, its verification provenance, and its boundary conditions. It enters the Layer 0 cache. The next time a `ConsistencySolver` evaluates a related claim, this record can become part of the topology it reasons against. This reflects a dual use of the content-addressing infrastructure: it holds both the

execution harnesses Solvers think *with* (reusable Sema patterns such as `SteelmanCheck#38b9`) and the verified knowledge Solvers think *about* (claim records such as `VerifiedClaim#a3f7`). Scaffolding and knowledge share one store but not one status: only recurring *strategies* are promoted to Sema patterns, while individual claims accrete as records. Among the protocols presented here, Truthseeking is the one whose verified-claim store is designed to expand through use.

The protocol must not replace one centralized oracle with another. One proposed design sketches a decentralized oracle-alignment layer: a network of `DataRecorderSolvers` would capture physical observations (sensor readings, transaction logs, satellite imagery) with provenance chains, while a separate network of `ParserSolvers` would transform raw data into structured claims. Separating the records from later interpretations can make some retroactive tampering detectable, but it does not establish the truth of the original observation. Reputation, staking, downstream verification, and anchored provenance are candidate incentives and audit mechanisms; whether they sustain accuracy under collusion or correlated error remains open.

Above this infrastructure, an `ObjectivitySolver` evaluates each claim along the objective–subjective spectrum. Claims that register as subjective are not rejected but reclassified—routed to a `ContestationSolver` that maintains multiple competing interpretations as first-class objects, each with its own provenance, evidence base, and confidence envelope. The acceptance gate at this layer does not require consensus; it requires that each competing account is internally consistent and grounded in independently verifiable evidence. To illustrate: “The central bank set the rate to 14.25%” is a verifiable fact that routes through the standard verification layers. “The rate is too high” is a normative judgment that the `ObjectivitySolver` reclassifies and routes to the `ContestationSolver`, where competing economic interpretations coexist as typed objects rather than collapsing to a single verdict. What emerges is not a single verified truth but a typed epistemic landscape: facts that have survived adversarial scrutiny, interpretations that are transparently grounded, and contested claims whose disagreement structure is itself informative.

6.8 The Creation Protocol: Decomposing the Concept of Making

In a single context, the act of making entangles interpretation, form, and implementation: the pressure to produce output crowds out interpretation of the original need, and the appetite for implementation detail overwrites the architectural framing that preceded it. The Creation Protocol decomposes the concept of *making*—what it means to bring something into existence—into sub-concepts corresponding to distinct cognitive faculties:

IntentSolver: The concept of *need resolution*. Resolves fuzzy intent into a typed `FrameSpec`. May invoke the Orthogonal Insight Protocol to fundamentally reframe the problem—the highest-leverage application of lateral reframing, since a corrected frame propagates downstream through every subsequent node.

FormSolver: The concept of *architectural synthesis*. Generates a `PlanBundle`. May generate alternative architectures from different coordinate systems.

RealizationSolver: The concept of *materialization*. Recursively spawns implementation sub-trees.

FitSolver: The concept of *contextual adaptation*. Selects representations optimized for adoption by a target context—a distinct cognitive mode from generation or optimization, since choosing the form that makes the right action easy requires audience modeling that interferes with both.

ViabilitySolver: The concept of *operational integrity*. Detects anomalies and feeds typed reports through the Feedback surface.

These are not stages in a pipeline but conceptual Solvers the CreationSolver routes among via evidence-gated decisions (Section 3.3). The routing is content-driven: ViabilitySolver detects that users abandon after first use; CreationSolver routes to IntentSolver, FormSolver, or RealizationSolver depending on where the evidence says the failure lives. A stakeholder objection that challenges the original framing routes to IntentSolver; a constraint violation routes back to FormSolver, not forward to RealizationSolver. When a Frame Error routes execution back to IntentSolver, the system recognizes that the frame has been outgrown.

The tree extends where complexity demands it: a FormSolver may create an ImplementationSolver, which creates a MatchingEngineSolver and narrower design, testing, and benchmarking children. The Creation Protocol's decomposition can then recur across a marketplace, essay, software system, building, or curriculum. Pathway Memory would let its shared conceptual Solvers accumulate experience to the degree that cross-domain compounding holds (Section 11).

Successful routing patterns may crystallize into a MarketplaceCreator hash that seeds later builds. The Creation Protocol can also apply to itself: a CreationSolver may build a new thinking protocol and generate its contract specification.

6.9 The General Problem-Solving Protocol

Applying the four-test algorithm (Section 2) to problem-solving itself yielded six proposed dimensions:

- **Scope** (what is the problem?): boundary definition, ambiguity resolution, decomposition into addressable sub-problems.
- **Evidence** (what do we know?): data gathering, source evaluation, uncertainty quantification.
- **Constraints** (what limits us?): resource bounds, physical laws, regulatory requirements, ethical boundaries.
- **Stakeholders** (who is involved?): affected parties, competing interests, power dynamics, consent requirements.
- **Mechanism** (how does the solution work?): causal structure, implementation design, component architecture.
- **Dynamics** (how does it change over time?): feedback loops, adaptation, degradation, unintended consequences.

Apply the four tests. Remove Scope—you are solving an undefined problem (*necessary*). Remove Constraints—your solution violates reality (*necessary*). Can you change Mechanism without changing Stakeholders? Yes—many mechanisms can serve the same stakeholders (*independent*). Does every problem in the range claimed by this protocol have all six? Designing a bridge, resolving a conflict, writing a law, curing a disease—the proposed answer is yes (*universal*). Addressing all six is claimed to provide a structured approach to any problem in that range (*complete*).

The mandatory-abstraction construction received this decomposition as theory context, not as persistent graph state, and derived its own direct Root composition before seeing the problem corpus. Mandatory invocation of those model-derived Root dimensions is a construction rule, not evidence of universal coverage; broad reuse without semantic contract repair likewise does not validate the six-part proposal. It remains a decomposition to test, not an empirical conclusion from the run.

The specialized protocols above are refinements of this general protocol. PURE is a deeper decomposition of Scope for evaluation contexts. The Creation Protocol is a deeper decomposition of Mechanism for making contexts. Ethical Reasoning is a deeper decomposition of the intersection between Stakeholders and Constraints for normative contexts. The general protocol is what the RootSolver uses when no specialized protocol fits. The specialized protocols are what it routes to when it recognizes a specific cognitive need that warrants deeper structure.

6.10 Meta Protocols: Decomposing the Concept of Self-Regulation

The preceding protocols operate within the solver tree. Meta Protocols operate *on* it—decomposing the concept of *self-regulation*: monitoring whether the tree remains the right shape for the problems it faces.

A MetaObserverSolver maintains a necessarily approximate representation of the tree’s topology and performance dynamics. It sees what no local solver can: redundant computation across branches, decompositions that have outlived their usefulness, pathway memory that has drifted. Its representations may be unstructured and lossy—the cost of maintaining the gestalt that typed decomposition sacrifices. Its value lies in the holistic pattern-matching that typed boundaries are designed to prevent within local nodes. Meta Protocols are the regulatory circuits of the solver network: they do not solve problems; they ensure the problem-solving topology remains adapted. Because a MetaObserver is itself a Solver, it may decompose its observation into sub-observers under the same marginal-value rule.

Three representative Meta Solvers: a TopologyAuditorSolver that detects when a subtree generates more rework than value (rising rejection rates) and emits a ReframeSignal; a RedundancyDetectorSolver that merges isomorphic sub-problems being solved independently across branches; and a DriftMonitorSolver that flags when pathway memory has drifted from current problem distributions.

A fourth, the OutcomeArbiterSolver, addresses the architecture’s most consequential evaluation problem: comparing solution quality across structurally different decomposition paths. It operates behind a hard seam, *blind to the decomposition that produced the result*, evaluating final artifacts against the original Task without access to intermediate structure, routing decisions, or gate history. This blindness prevents contamination by structural familiarity or sunk-cost reasoning. The difficulty is that solution quality for open-ended problems is not fully objectifiable. The architecture addresses this by separating the empirical layer (what measurably happened: cost, completion, constraint satisfaction, downstream gate rejection rates) from the normative layer (was the result *good*). The empirical layer is objective and automatable. The normative layer requires a model whose evaluative judgment can be trusted—an evaluator whose values are the substrate of cognition rather than a post-hoc filter [5], structurally reducing the sycophantic drift that undermines evaluators trained on capability first and judgment second. The OutcomeArbiterSolver’s verdicts flow back to the RootSolver’s Pathway Memory as typed RouteComparisonSignals: for this problem class, route A scored x on the empirical layer and y on the normative layer; route B scored x' and y' . Over time, these build the library of proven routing decisions—the bridge between execution and compounding.

7 The Cognitive Commons

Because the proposed Solver architecture applies the same contract at every scale (Section 4.3), it could extend into a permissionless, planetary-scale network: a *cognitive commons* where specialized Solvers could think together across disciplines without requiring a single coordinating institution. Such a commons would function as a library of persistent abilities—question-structures that have been decomposed, verified, and made available to later intents. Bostrom [60] categorized a related vision as *Collective Superintelligence*—the amplification of collective performance through improved coordination rather than individual capability. The Cognitive Commons pursues a similar goal but through cross-organizational composition and typed contracts rather than the top-down organizational enhancements Bostrom envisioned; whether it achieves superintelligent-level capability remains an open question contingent on the compounding hypothesis (Section 11).

A structural property explains how this could decentralize functions that conventional organizations keep internal. Human institutions organize by *role*, rationally, because bounded minds cannot each hold the whole structure (Thesis 7); but a role has no identity outside the firm that confers it, so a hierarchy of roles cannot leave the building. A hierarchy of concept-defined Solver specifications can cross organizational boundaries: once adopted, a specification's identity is fixed by its content rather than by an organization, so the same `DistributionProtocolSolver` specification can be reused in economics, agriculture, and diplomacy. Some organizational functions could therefore be unbundled into a commons whose unit of organization is shareable by construction. Institutions would not thereby become dispensable: they also supply authority, responsibility, rights, and legitimacy.

In its open form, such a commons could grow through accumulation rather than central design. One problem-solving service might first retain structures developed across its cases. Several services could then share selected structures, specialize around recurrent capabilities, and expose those capabilities as Solvers to a wider network. Firms, public institutions, and physical infrastructure could join within their own permission and authority boundaries. If adoption and compounding proved useful, the result could be a cross-organizational problem-solving substrate rather than a single organization, canonical tree, or central authority.

7.1 Governance as a Cognitive Constitution

If such a commons became infrastructure through which institutions routinely framed problems, allocated reasoning, and coordinated action, it would do more than connect Solvers. Functionally, it would begin to act as a *cognitive constitution*: a system that determines how collective questions become legible, which constraints bind their resolution, where attention flows, and what prior reasoning the next decision inherits. At that scale, the enduring cognitive entity would no longer be any single model or institution, but the revisable topology through which successive models, people, and organizations reason. The analogy is not legal: the architecture supplies neither constitutional legitimacy nor a prediction of inevitable planetary adoption. It identifies the governance implications of persistent conceptual coordination at civilizational scale.

The constitutional functions are distributed across the architecture. The root Task sets the agenda: it defines which condition is treated as a problem and what a satisfactory resolution would mean. Decomposition supplies the frame, making some dimensions explicit while leaving others outside the cut. The four tests discipline that act; they do not make it culturally or normatively neutral. Acceptance gates specify which artifacts may cross a seam; at hard seams, they also define what no downstream gain may compensate for. Routing allocates scarce attention, compute, expertise, and—in an economic commons—payment. The serving layer supplies precedent: recurrent, validated, or niche-leading structures promoted from the archive become the inexpensive, familiar paths through which later problems are interpreted, while the inclusive archive retains alternatives and negative

knowledge. The OutcomeArbiterSolver performs a limited form of judgment by comparing routes against the original Task. Each function is local, but their composition governs the space of collective reasoning.

At scale, persistence, differential reuse, and outcome-informed routing could produce evolution-like selection: carves and routes supply variants; gates, costs, and outcomes shape reuse; reframing and composition generate more. If scarce tasks, compute, or payment are allocated among them, the commons could become an *evolutionary market for intelligence*, where reusable cognitive structures compete for trials. Monetary exchange may influence selection but is not required.

The environment would be institutional: root Tasks direct attention, AcceptSpecs and outcomes define success, and routing and budgets allocate trials. Prevalence might reflect adaptation, lock-in, capture, or chance, while traffic compounds early advantage. Archives, protected exploration, and plural routing mitigate premature convergence; they do not guarantee diversity.

Technical decentralization does not by itself decentralize political power. Content addressing can establish that two parties invoke the same immutable specification; it cannot establish that the specification is correct, legitimate, or appropriate. A nominally permissionless network could still concentrate control around dominant RootSolvers or routing policies, widely adopted AcceptSpecs, serving-layer curators, or the owners of compute and outcome data. Provenance makes those choices more inspectable, but inspectability is not consent. A commons can therefore be decentralized in topology while remaining centralized in agenda, framing, judgment, or resource allocation.

Operational trust is a separate layer: cross-organizational composition has no universal RootSolver to impose one AcceptSpec or settle disputes. Solvers may submit plausible garbage for bounties and orchestrators may fabricate rejections. The Receptivity Gate (Section 5.2) and non-compensatory acceptance gates can expose some such failures as typed, auditable events; they do not establish truth or confer political legitimacy.

At that scale, the people affected by a decomposition may not be the people who submitted the Task, and a cleanly typed frame can still omit what they regard as essential. The unresolved requirements are constitutional: the ability to challenge a goal or carve and to appeal a gate decision or arbiter verdict rather than merely retry within them; meaningful exit and the ability to fork an inherited structure; plural RootSolvers, routing policies, and jurisdictions; independent audit; and rules for versioning or superseding, expiring, quarantining, or—where rights or safety require—deleting persistent structures. Recoverable alternatives support political pluralism only when actors can discover, fund, and route through them. The archive and content addressing provide mechanisms for recovery and forking, but the architecture implements none of these as enforceable rights. It reveals where those rights would have to attach.

Prediction and decision markets could be optional sensors, not sources of governance or legitimacy. They might forecast demand, failure, or outcomes conditional on a route, echoing futarchy's separation of values from market-elicited beliefs [61]; the Iowa Electronic Markets show useful forecasting in one repeatedly resolvable setting [62]. But popularity is not value, price-guided routing can help realize its own forecast, and some deterministic maximum-probability rules can reward exaggeration [63]. Because unchosen routes hide counterfactual outcomes, strict properness in Chen et al.'s model requires nonzero selection probability for every route [64]. Market outputs should inform rather than determine routing after goals and rights are specified elsewhere.

Within a legitimately governed jurisdiction, gate histories, provenance, costs, and outcome records can still inform routing and compensation. Consider pandemic preparedness: an epidemiological Solver in Geneva passes an outbreak forecast to a supply-chain Solver in Singapore, which passes a resource plan to a policy Solver in Nairobi. At each seam, the parties authenticate the contract and provenance and test the declared criteria; different branches recurse only while their expected gains justify their incremental cost. These checks reduce ambiguity and make responsibility more inspectable. They do not by themselves establish the forecast's truth, the policy's legitimacy, or the

authority to enact it. The architectural aim is a strategy whose epidemiological, logistical, and policy assumptions can be evaluated separately and then composed.

7.2 Knowledge Sharing Models

A critical design question arises when multiple organizations deploy Solvers defined by the same concept. Company A has an `IncentiveAlignmentSolver` that has accumulated Pathway Memory from financial services problems. Company B has an instance of the same concept that has learned from healthcare. Whether, and at what layer, they should share what they have learned is a design question. Three models span the design space:

Open commons. Every Solver shares its Pathway Memory with every other instance of the same concept. Company A's financial services insights immediately benefit Company B's healthcare instance. This maximizes the pool of available signal—the trunk can thicken fastest when everyone contributes—but also expands privacy, capture, negative-transfer, and common-mode risks.

Federated sharing. Solvers share structural insights—which sub-concepts work, what depths are productive, which decompositions fail—but not domain-specific data. Company A learns that `IncentiveAlignmentSolver` decomposes better into intrinsic, extrinsic, and structural dimensions than into individual and collective. That structural insight transfers without revealing proprietary financial services data. The shared architectural knowledge improves while the competitive advantage from domain-specific application is retained.

Proprietary isolation. Each organization keeps its Solver instances entirely private. Company A's version is tuned to financial services—competitive advantage preserved. But it never benefits from Company B's healthcare insights, and vice versa. This reduces pooled learning, but may rationally preserve privacy, local sovereignty, and ecosystem diversity.

The tension resembles open-source and proprietary software: conceptual structure may be shared infrastructure while domain-specific Pathway Memory remains private. A Sema hash identifies the immutable decomposition, gates, and sub-solver types; organizations can share that conceptual skeleton while building and retaining their own mutable experience through use.

7.3 Task Deduplication

Cross-organizational execution could also support task deduplication by extending content-addressing to a canonical Task payload. If orchestrators encode every semantically relevant input, constraint, and `AcceptSpec` reference into byte-identical canonical payloads, those payloads receive the same identifier. A routing layer could then merge them: one specialized `ChildSolver` executes the work once, while each parent evaluates the shared `Result` against its own continuation. Mathematical similarity alone is insufficient, and semantic equivalents encoded differently will not collide. Active-compute deduplication is therefore a proposed optimization contingent on canonicalization, authorization, and compatible acceptance scopes, not an automatic consequence of hashing.

If two tasks require different implicit contexts, those differences must be typed into the inputs or constraints, producing different hashes and gracefully separating their execution. This is the mechanism's central challenge: implicit context is implicit precisely because the agent may not know to type it, risking false deduplication when tasks appear identical but carry unrecognized contextual differences. Sema's hash granularity—which extends below the task level to individual constraints and acceptance criteria—mitigates but does not eliminate this risk; robust solutions likely require anomaly detection on result divergence when a shared solver's outputs satisfy some parents but not others.

7.4 Beyond Computation

The tree need not stop at computation. Because the contract is substrate-agnostic, physical Solvers participate on the same terms: a pharmaceutical factory whose production line exposes the Solver interface accepts a typed drug-manufacture task, returns a typed result (batch purity, yield, provenance), and is subject to the same acceptance gates as any computational node. Drug discovery, clinical verification, manufacturing, and distribution become branches of a single solver tree spanning laboratories, factories, and logistics networks, each node governed by the contract, whether it runs on silicon or steel. Once a Solver can actuate factories, infrastructure, or resource allocation, authorization must remain explicit at the action boundary: local gate clearance does not settle political legitimacy or responsibility for distributed harm.

7.5 Persistence

The archive-and-serving policy of Section 2.10 extends across organizations. “Global” means addressable within the relevant trust and permission boundary, not that private payloads must be public. The resulting topology could let distributed human and machine expertise compose across disciplines without requiring centralized retraining. One distribution model is world-specialized models exposed as reusable *thinking protocols* rather than only as general-purpose agents: task protocols coordinate what to do next, while thinking protocols preserve a reusable way of reasoning about a recurring conceptual class. Orthogonal Insight sketches one such operator [1]. Some problems are intractable because available expertise cannot be composed; the commons is intended to provide that substrate.

The cognitive commons remains a long-term vision rather than a demonstrated system. Scaling the proposed decomposition, contract, and persistence mechanisms to that horizon would leave the constitutional questions above unresolved, along with operational questions such as certification, malicious compliance, semantic versioning, and how constraint monotonicity and AcceptSpec negotiation work without a shared RootSolver. Market-mediated cross-organizational composition would also need compensation and settlement for accepted work. One possible mechanism is a utility-token protocol with gate-contingent bounties; such a mechanism could coordinate payment but would not confer legitimacy.

8 Growing the Tree Through Training

The following training regime is theoretical and has not been implemented. One possible deployment would compose pre-trained, independently capable models as nodes behind contract boundaries, then fine-tune them jointly through the contract, use gates as training signals, learn routing, and allow topology to grow through use.

Existing multi-agent training systems implement pieces of this. MALT [65] trains a generator, verifier, and refiner jointly with credit assignment—a fixed three-node pipeline. Generative adversarial networks co-train a generator and discriminator through adversarial feedback. Multi-agent reinforcement learning [66, 67] trains populations of agents with individual rewards in shared environments. The canonical FedAvg form of federated learning [68] aggregates updates from heterogeneous nodes into one shared model, with subsequent work extending federated training to LLMs; the proposal here instead preserves role-specific models and exchanges selective gate signals rather than averaging their parameters. Multi-agent debate [69, 70] composes multiple LLMs at inference time and performs no weight updates. Co-distillation and online mutual learning [71, 72] pair models through soft-target supervision that encourages representational agreement; the proposed gate signals would instead let roles specialize behind separate objectives. Since the first release of this paper (April 2026), the portfolio-of-specialists premise has reached production scale: DeepSeek-V4

trains domain experts—mathematics, coding, agentic use, instruction following—independently through supervised fine-tuning and reinforcement learning, then consolidates them into a single student via on-policy distillation [73]. The specialists are carved by inherited task domains and absorbed rather than retained as reusable infrastructure. Among the systems reviewed here, none jointly trains a growing tree of heterogeneous LLMs through typed gate signals while splitting its topology in response to performance divergence. The Solver Contract is proposed as infrastructure for that experiment.

8.1 Joint Training, Isolated Weights

The proposed core shift is to treat the outermost gate as the system-level reward while updating weights locally: each node would fine-tune, through LoRA [53] or an equivalent method, on gate outcomes produced by the system as a whole.

At the systems level, one useful shorthand is: the gate is the local loss, the topology is the computational graph, and Pathway Memory is the routing optimizer. This is an analogy, not an implemented equivalence. Gates emit discrete and possibly mistaken judgments, the topology is not assumed differentiable, and neither supplies the missing real-world outcome signal.

The gates sever shared loss *across* nodes, not local optimization *within* them. Each node is a standard LLM whose internal weight updates use standard gradient methods. The gate tells each node what to learn from—which outputs were accepted, which were rejected, and why—and the node fine-tunes on that signal using conventional techniques (supervised learning on accepted outputs, DPO or RLHF on accept/reject pairs). Each node is fine-tuned on its specific role: leaf nodes learn to solve their assigned subproblem class, parent nodes learn to decompose and route, and gate evaluators learn to judge. The training is “joint” because all nodes are fine-tuned concurrently on signals produced by the system as a whole, although gradients do not flow across the tree.

Unlike Mixture-of-Experts [74, 75, 76], whose experts participate in a shared end-to-end objective, Solver nodes in this proposed regime would exchange only gate signals across their boundaries. The collapse of modular architectures under end-to-end training [77] motivates this harder separation. The compositional objective asks whether the routed nodes clear the outermost AcceptSpec, in contrast to Soft MoE [78] and Expert Choice routing [79], which remain differentiable or are trained through a shared objective. The non-differentiable gate supplies a hard interface discontinuity intended to reduce cross-node objective interference; it does not by itself guarantee cognitive isolation.

8.2 The Reward Signal

The compositional objective still requires ground truth. In verifiable domains—mathematics, code, logic—proof checkers and tests provide objective reward. For open-ended problems, the outermost gate must learn from human-judged examples, as in RLHF; the blind OutcomeArbiterSolver of Section 6.10 formalizes this empirical–normative boundary.

A potential advantage over single-model RLHF is more localized credit evidence. When a monolithic model produces a poor output, the reward signal must propagate through billions of undifferentiated parameters. When a solver tree produces a poor output, each internal seam has its own rejection rate, each node appears on successful and failed paths, and routing decisions can in principle be compared against alternatives. Nodes with high rejection rates and nodes common to successful paths become candidates for investigation, but these correlations do not establish causal credit. Counterfactual routing is required to distinguish bottlenecks from inherited upstream failures.

8.3 Bootstrapping from Human Decomposition

The regime above presumes that its models can already decompose problems adequately. An alternative bootstrapping path would distill decomposition traces from human experts. Many independent expert decomposers, working separately on different problem streams, produce traces from which recurring substructures can supply candidate invariants; divergent carve families are retained when blind outcome evaluation shows complementary utility. Human involvement could taper as the learned decomposer generalizes, without requiring the initial model to possess a general decomposition faculty.

Cross-decomposer agreement measures whether particular structure is founder-independent (Section 11), but disagreement is not automatically noise. Averaging incompatible carvings could produce a decomposition that fits no one; erasing every minority carve could discard a niche leader. Training should therefore learn shared atoms and a conditional router over useful alternatives rather than one averaged tree. A capable model could later bootstrap the same process itself.

8.4 Bootstrapping from the System’s Own Derivations

The preceding subsection closed with a possibility: a sufficiently capable model could eventually bootstrap the decomposition process itself. The archive is what would make that concrete. Every attempted problem deposits an evaluated solution graph: the carve chosen, the routes traversed, the gate verdict at each seam, and the branches tried and abandoned along the way. Taken together, these graphs would form a training corpus for the decomposition faculty itself, distinct from the per-node fine-tuning above. Rather than each node sharpening within its isolated role, a single model would train on the archive’s accumulated derivations so that the method—carve, gate, deepen or stop, reframe—amortizes into its weights—a proposed *deliberation–amortization cycle*.

Two properties would distinguish this corpus from ordinary synthetic data. First, it records executed computation rather than generative imagination. The search was actually run, and its rejected branches are not noise to filter out but the calibration signal: they encode where carves of a given shape fail, and where the marginal-value rule’s correct verdict was not to decompose at all (Section 3.2). Training only on successful paths would reproduce the survivorship bias inherent in published reasoning. The nearest existing evidence comes from constrained symbolic domains, where models trained on full search trajectories, errors included, outperform models trained on optimal paths alone [80]. The typed, evaluated graph is a richer object than the flattened strings used there: rejections name the violated `AcceptSpec` clause, and abandonment carries provenance. Production post-training now consolidates specialist teachers into one student by imitating their outputs [73]; the corpus proposed here differs in recording the search rather than the answer. Second, the corpus is grounded in gate verdicts and, at the outermost boundary, in oracles or human judgment rather than in the generating models’ own output distribution—the closed-loop condition implicated in model collapse [81].

The next subsection develops a spectrum for routing, from explicit reasoning-capture graphs to weight-encoded intuitions. A model trained on the archive would extend that spectrum to the faculty as a whole and occupy its implicit end: the tree internalized, applying the carve-and-gate discipline within a single context window, and instantiating the physical architecture only where a problem’s depth demands it. Template retrieval at inference [82] occupies the explicit end—structural reuse without weight amortization. The internalized form carries its own low-cost falsification: on problems below the decomposition threshold, a derivation-trained model must match its base model’s accuracy at equal or lower token cost. Systematic over-decomposition of trivial problems would show that the corpus taught the recursive case without its base case—the depth-zero carve, where the calibrated action is direct execution.

8.5 Routing, Topology, and Co-Evolution

Every parent Solver that makes routing decisions—including the `RootSolver`—is itself a model being fine-tuned on routing quality. Its weights encode which paths work for which problem types. The routing model, the leaf specialists, and the gate evaluators are all learning simultaneously. Each shapes the others because each one’s training signal depends on what the others do.

Pathway Memory exists on a spectrum. At the explicit end, a reasoning-capture graph [6] stores structured cognitive history—inspectable, debuggable, transferable. At the implicit end, the routing model’s weights encode routing intuitions at inference speed. A production system may use both: explicit graphs for cold-start bootstrapping, implicit weights for sub-millisecond routing decisions.

Under joint training, the topology itself becomes a learned parameter. The tree begins as a minimal seed. As gate signals accumulate, the system discovers where specialization is needed. A node whose rejection rate diverges across problem classes signals that a single generalist is doing two jobs—and the split that follows is a consequence of the training dynamics, not a design decision. In standard deep learning, the architecture is fixed and the weights learn. Neural Architecture Search [83] learns the architecture as a separate outer loop. In a jointly trained solver tree, architecture and weights co-evolve in a single process: nodes split because the weights reveal they are doing two jobs, and the routing model rewires because specialists have become good enough to justify a new path.

The biological parallel is instructive. The brain does not begin with its adult connectivity; it begins with massive synaptic overproduction, then prunes based on use [84]. A jointly trained solver tree follows the same principle—except that it can also grow new specialist nodes, a capacity biological development largely forecloses after early critical periods.

8.6 Credit Assignment Through Structure

A caveat limits structural credit assignment: if plausible upstream garbage passes its gate, a downstream node may receive the rejection for corrupted input. These signals are sparser than back-propagation. Local signals are therefore clues rather than proof of fault; counterfactual execution over alternative routes is needed to isolate cascading errors. This connects to QMIX [85], which decomposes global reward into agent contributions; here the nodes are the agents and the outermost `AcceptSpec` supplies the global reward.

The joint-training regime described in this section has not been implemented. As formulated, it assumes discrete models communicating through API boundaries; if implemented, it could point toward a longer-term hardware horizon: *solver-native models*. Ultimately, the hard acceptance gates and contract boundaries described here could compile into geometric constraints in latent space—verifying that intermediate activations fall within valid conceptual manifolds during a single forward pass, replacing serialized text handoffs with dynamically routed, non-differentiable internal boundaries.

9 Mandatory-Abstraction Construction: Structural Conformance

The preceding sections describe the architecture theoretically, the protocols illustrate it through design proposals, and the commons and training sections project it forward. This section reports the canonical planning artifact built to probe structural preconditions of the theory. A GPT-5.6 Sol construction asks whether mandatory upward abstraction can produce one rooted hierarchy, followed by conceptual decomposition and persistent reuse, when only `RootSolver` is supplied as authored persistent graph state. Without common-node traffic and coherent abstraction routes, no shared cross-problem signal could accumulate at recurring nodes and no route could become a reasoning

highway. This is a construction-conformance probe, not a performance experiment. A superseded Gemini artifact is retained at the end of the section solely for reproducibility and error provenance.

Evidence boundary of the construction artifact

The artifact establishes that model-authored node and edge proposals can be persisted, routed, reused under several parents, and checked for declared structural invariants. It exposes prompt-conditioned topology and semantic failure cases. It does *not* establish that the proposed concepts are true, necessary, sufficient, or contract-preserving; that child Solvers executed independently; that reuse transferred competence; that Pathway Memory learned; or that any solution improved. Counts below describe structurally recorded proposals, not semantic validation or outcome performance.

9.1 Construction Method

The artifact uses the 100-problem, 20-domain corpus released with the earlier study. Its only authored persistent node is RootSolver. Before seeing any of the 100 problem texts, one GPT-5.6 Sol call at medium reasoning effort received the theory prompt—including the four tests, the economy and restaurant examples, and the proposed General Problem-Solving Protocol—and derived 24 non-root nodes. Six are direct Root composition dimensions: Problem Constitution, Explanatory Understanding, Resolution Synthesis, Resolution Commitment, Resolution Realization, and Resolution Regulation. The scaffold is therefore model-derived and problem-blind, but it is theory-conditioned rather than taxonomy-unprimed.

Six further low-effort calls process batches of 10, 20, 20, 20, 20, and 10 problems. Calls are ephemeral and neither resume nor persist a CLI conversation; the controller rehydrates the current node catalogue and typed edges for each batch. For every case, the model must construct a specific-to-Root specialization ascent, perform parent and sibling search at every attachment, traverse the exact reverse route, invoke the six direct Root dimensions, decompose the specific subject into at least two composition children, and declare how those children synthesize the parent’s outward capability. Nodes can receive several parents with different edge-local roles. Deterministic code checks references, edge types, one Root-reachable acyclic graph, exact route reversal, mandatory scaffold use, and atomic checkpoints. It cannot check whether a category is true, useful, complete, or contract-compatible.

9.2 Structural Results and Failure Analysis

The completed artifact contains 311 nodes and 603 unique typed edges: 441 composition and 162 specialization. All 100 cases are structurally accepted, every node is reachable from the single Root, and the structural audit finds no cycles, dangling references, duplicate identifiers, duplicate typed edges, or relation conflicts. Excluding the 700 mechanically forced invocations of Root plus its six direct dimensions, the case records contain 500 reuses and 286 creations (63.6% reuse). Of 417 thing-specific constituent invocations, 290 reuse an existing node (69.5%); 99 of the 286 problem-created nodes recur, 66 across domains, and 78 receive more than one persistent parent. Later cases also create higher parents over earlier nodes: for example, Problem 12 places the earlier Educational Strategy Game Design and a new cooperative game capability under Strategy Game Design, which later routes three further game cases and one diplomacy case. These are structural, model-proposed reuse counts, not verified semantic equivalences.

The same audit exposes the principal failure. The model makes zero contract revisions even when reuse widens beyond the first domain. Evidence Traceability, born with a newsroom/publication boundary, becomes a child of 18 parents across 10 domains without repair. Other reused nodes retain similarly narrow first-domain language, and Talent Development System Design is placed beneath

Organizational Capability Transformation even when later cases concern childhood regulation, literacy, fermentation, and mentorship. Thirty-six of 59 new abstraction parents currently serve one case, 187 problem-created nodes remain single-case, and only 4 of 18 deeper bootstrap nodes are ever invoked. The run therefore demonstrates that mandatory upward-first construction can create a rooted persistent DAG with nontrivial reuse and declared nested composition; it does not establish ontology correctness, independent child execution, operational emergence, learning, or outcome gains.

The run is an adaptive construction artifact, not a frozen benchmark. Local pilots used the same corpus; after the first ten accepted cases, checkpoint size changed from 10 to 20 and public-output verbosity targets were shortened, although the semantic construction rules and structural acceptance checks were retained. Each batch call could see all texts in that batch, so instructed no-lookahead is not independently auditable. The seven model calls took 3,157.211 seconds (52m 37.2s), and the state interval was 56m 02s; the model-call total missed the 30-minute target by 22m 37.2s. No semantic reviewer calls or private reasoning traces were retained.

9.3 Superseded Seeded Gemini Archive

An earlier Gemini 3 Flash artifact processed the same corpus through a different planning harness. A retrospective audit established that an implementation error initialized it with an unintended 29-node persistent scaffold: the six proposed dimensions of the General Problem-Solving Protocol plus 23 children generated before the first problem. The runner also protected the top-level seeds. Of 606 saved cross-domain reuse actions, 483 (80%) subsequently routed through those initial nodes. The planner described child outputs inside single model calls rather than executing independent child Solvers, while structural checks logged conflicts without enforcing repair. Its reuse figures therefore cannot evidence independent discovery, operational emergence, nested execution, learning, or solution quality.

The complete artifact remains public as a superseded diagnostic record because its failure is scientifically useful: supplied abstract structure can attract substantial traffic and imitate discovered reuse. It motivated the corrected mandatory-abstraction construction above, whose only authored persistent node is `RootSolver`. The two artifacts use different models, prompts, ledgers, and controls and are not a performance comparison; the earlier artifact is retained only for reproducibility and error provenance.

9.4 Experimental Reference Implementation

A separate alpha reference protocol and implementation for the proposed Solver architecture, *Fractal Intelligence Protocol*, provides a centralized coordinator and node SDK together with experimental modules for selected parts of that architecture [86]. Version 0.2.0 implements a bounded, oracle-driven four-test procedure and marginal-value recursion; replay-safe typed delegation and recursive task trees; deterministic non-compensatory gates; content-addressed accepted-artifact retention; and a five-arm matched-budget harness. It also adds a review-only topology profile for mandatory specific-to-Root ascent, genus-and-differentia sibling records, exact reverse routes, immutable rooted-DAG patches, typed specialization and composition edges, multi-parent edge-local roles, and explicit parent synthesis. Approved direct composition work can compile into the existing backend-neutral execution plan while passive specialization membranes remain routing context. Deterministic validation establishes artifact consistency, not semantic truth, and the profile does not grant coordinator, admission, or payment authority. There remains no production model adapter, autonomous Frame Error detection, semantic contract repair, learned Pathway routing, or external settlement, and the software provides no evidence of improved solution quality or compounding reuse.

Table 5: Mechanism status in *Fractal Intelligence Protocol* v0.2.0. “Implemented” means present in the artifact with deterministic tests; it does not imply semantic or outcome validation.

Mechanism	Implemented or specified	Current evidence boundary
Contract boundary	Manifest and Execute transport; typed Task, Result, and deterministic AcceptSpec records	Consult, Verify, and Feedback remain reserved transport surfaces; structural conformance does not prove behavioral substitutability
Decomposition	Oracle-driven four-test control flow, bounded completeness search, routing probes, and marginal-value recursion	Deterministic fixtures exercise control flow; the oracle’s semantic judgments are not mechanically verified
Topology	Mandatory ascent, sibling records, reverse routes, typed edges, immutable patches, multi-parent roles, and synthesis records	Validators prove rooted-DAG and record integrity, not that a parent, carve, reuse, or synthesis is conceptually sound
Execution	Replay-safe delegation, recursive task trees, deterministic gates, content-addressed accepted artifacts, and backend-neutral Solo plans	No production model adapter or comparative outcome result; probabilistic evaluation remains untrusted
Reframing/learning	Administrator-authorized root reframe and retained pathway/gate records	No autonomous Frame Error attribution, semantic repair, learned routing, or demonstrated compounding
Evaluation	Five matched-budget arms, blind-evaluator interface, and call/token/cost accounting	Scripted examples validate the harness; no arm has yet established a performance advantage

The accompanying repository publishes the alpha’s versioned protocol, architecture, conceptual-decomposition, topology-construction, Solo-mode, benchmarking, and verification documents together with its tests. Those files supply implementable schema and state-transition detail for the mechanisms listed as implemented above; they are not a complete specification or validation of the broader five-surface architecture.

10 Related Work

This paper draws on and departs from six adjacent literatures: conceptual decomposition methods, boundary formalization between agents, cognitive architectures for specialization, generator-verifier pipelines, outcome-grounded learning, and economic governance of compute. The central question for evaluating related work is relational rather than terminological: does a framework partition a chosen procedure into actions, stages, or roles, or propose constitutive capabilities through which several procedures and instances can route (Section 2.1)? Persistence and abstract labels alone do not decide the question. Each body of work is evaluated through this lens.

10.1 Conceptual Decomposition Methods

The algorithm proposed in Section 2 has identifiable ancestors. Zwicky’s morphological analysis [22] is the closest: it decomposes a problem into independent dimensions and systematically explores the combinatorial space. Zwicky applied this to jet engine design, satellite systems, and astrophysical classification—demonstrating that dimensional decomposition produces solutions invisible to conventional approaches. However, morphological analysis decomposes *design parameters* (fuel type, nozzle shape, combustion method), not the conceptual structure of what a domain is made of, and the method is flat rather than recursive.

Gärdenfors’ conceptual spaces framework [87] provides a theoretical foundation: concepts are represented as regions in quality dimensions, and reasoning about concepts is reasoning about geometric relationships in these spaces. The four-test algorithm seeks candidate quality dimensions of this kind. Gärdenfors describes the structure of concepts but does not provide this recursive generation-and-persistence mechanism.

Dijkstra’s separation of concerns [88] provides the independence principle that governs modular decomposition in software engineering. Parnas [13] operationalized this for information hiding. Both apply to code modules rather than cognitive concepts, but the structural insight is identical: good decomposition minimizes coupling across boundaries.

The consulting principle of MECE (Mutually Exclusive, Collectively Exhaustive), widely attributed to McKinsey, maps to the independence and completeness tests. It lacks the necessity test (which asks whether sub-concepts are load-bearing rather than merely categorically clean) and the universality test (which searches beyond instance-specific categories across the range currently claimed). And it does not recurse.

Hierarchical task networks are the canonical *task*-decomposition tradition: HTN planning [89] and its planners (SHOP2 [90]) recursively refine tasks into subtasks via domain-specific methods. The proposed contrast is the concept–task distinction of Thesis 1: HTN methods encode *how to do* a task in one domain, while the four-test operator searches for constitutive dimensions of *what capability is being realized*. Neither hierarchical form nor abstract naming proves that distinction; the matched comparison must test whether the resulting units preserve their contracts and transfer beyond equally persistent task methods.

Persistent reasoning reuse is a concurrent theme. Graph-Memoized Reasoning [23] stores reasoning workflows as graph-structured memory, encoding past decision graphs and retrieving them through structural similarity to enable compositional reuse of subgraphs across tasks. Dep-Search [91] learns dependency-aware reasoning traces with persistent memory. Intern-S1-MO [92] maintains a lemma library for reusing historical conclusions across mathematical reasoning steps. All three persist *execution traces* (the steps an agent took to solve a problem) and retrieve similar step-sequences when a new problem looks structurally similar. The proposed reuse mechanism differs: reasoning highways would emerge if independent application of the four tests to unrelated domains converged on common sub-concepts. That convergence remains an empirical prediction, not a result of the mandatory-abstraction artifact, which records declared reuse within one theory-conditioned construction. Both traditions share the broader engineering principle of retaining reasoning structure.

Library learning comes closest to the compounding claim. DreamCoder [93] alternates solving problems with abstracting recurring program fragments into a growing library, demonstrably compounding—but within a single domain-specific language and task distribution, with no typed boundaries, no economic depth governor, and no cross-organization identity. Voyager [94] accumulates a skill library for an embodied agent, persisting *task* skills rather than conceptual structure. Shared upper ontologies (Cyc [95], SUMO [96]) are the prior art for concept infrastructure intended for cross-domain reuse—including the synonymy and alignment problems Section 11 inherits—and their history of under-delivered reuse is the cautionary precedent: this architecture bets that reuse must be *operational* (solvers invoked at runtime, selected by routing evidence) rather than declarative (assertions consulted), and the reproduction test is what would distinguish the bet from a repetition of that history.

10.2 Boundary Formalization

Agent communication has a formal lineage: KQML [97] and FIPA ACL [98] drew on speech-act theory to type inter-agent messages by performative intent (inform, request, propose, commit) rather than by payload alone—an early recognition that the semantics of the boundary, not just its schema, is load-bearing. The foundational work on contract-based coordination is Smith’s Contract Net Protocol [18], which formalized negotiation-based task distribution through task announcements, bidding, and contract awarding. The Solver Contract inherits the structural intuition (typed agreements between autonomous agents) but adds cognitive mode declaration, acceptance gates that can return typed restructuring requests, and persistent decomposition.

The 2024–2026 wave addresses structural interoperability: MCP [99] standardizes tool access, A2A [100] introduces Agent Cards and task lifecycle, ANP [101] provides decentralized discovery, OpenClaw [102] composes these into a persistent runtime with multi-agent routing. DSPy [19] introduces typed *signatures* for language model programs, the closest existing construct to the Solver’s typed surfaces, but signatures describe input-output shape, not cognitive mode or acceptance criteria. GPTSwarm [20] models agents as recursively composable optimizable graphs, directly paralleling fractal composition, but optimizes the graph topology for task performance rather than decomposing the conceptual structure of what an ability is made of. Role-based multi-agent frameworks such as MetaGPT [103] encode standardized operating procedures into agent pipelines, and Mixture-of-Agents [104] shows that layered aggregation of heterogeneous LLMs improves output quality; both compose agents by task role rather than by conceptual decomposition with typed cognitive mode. All coordinate what agents do; none governs the cognitive mode at each node or persists the decomposition as reusable structure.

Three concurrent works apply formal contracts to agents. A Design-by-Contract neurosymbolic layer [105] wraps individual LLM calls in Hoare-style conditions—single-surface contracts on individual calls. Agent Contracts [21] proposes a 7-tuple framework ensuring resource conservation with 90% token reduction and zero conservation violations. Agent Behavioral Contracts [106] extends to session-level behavioral specifications with a compositionality theorem. Both primarily treat the boundary as a validator. The Solver Contract addresses a complementary cognitive contract: the Consult surface enables depth estimation, the Feedback surface supplies localized evidence via the Receptivity Gate, and the Manifest declares conceptual mode. Its central hypothesis is that typed rejection can make the boundary an intelligence contributor by carrying a structural reframing request upstream rather than merely blocking an artifact (Sections 4.6 and 3.4). POLARIS [107] provides converging industrial evidence that typed boundaries can improve agentic reliability. In a complete deployment, these contracts could be complementary: Agent Contracts governs budgets, ABC enforces behavioral invariants, and the Solver Contract structures conceptual composition.

10.3 Cognitive Architectures and Specialization

The actor model [15] supplies a foundational precedent for self-similar composition: every actor communicates through a uniform boundary, with no ontological distinction between a coordinator and a worker. That is the direct ancestor of the claim that a leaf Solver and a thousand-node subtree can present the same outward contract. Actors do not, however, supply conceptual decomposition, acceptance-driven restructuring, or persistent reasoning reuse.

The most direct organizational ancestor is the fractal manufacturing tradition. Warnecke [16] proposed self-similar, autonomous organizational units for factory floors; Ryu et al. [17] formalized these as agents with five internal modules (observer, analyzer, organizer, resolver, reporter). The structural parallel, self-similar units with uniform internal structure, is real. The distinction is equally precise: FrMS modules are an internal processing pipeline describing what happens *within* each unit; the Solver’s five surfaces are a compositional interface describing how units present themselves *to the outside*. FrMS decomposes manufacturing tasks directly into procedural units; this paper decomposes the conceptual structure of what an ability is made of, and tasks route through that structure. The proposed additions are the four-test algorithm, acceptance gates that can request restructuring, and content-addressed persistence.

CoALA [108] provides the standard decomposition of a language agent’s internals into memory, action spaces, and decision procedures. Wray et al. [109] map cognitive design patterns from Soar and ACT-R onto LLM agents. Both decompose what an agent’s components are; this paper decomposes what concepts interact across agents via typed boundaries. Monolithic models can compromise across competing objectives: the Safety Tax [8] reports reasoning degradation under

safety alignment. MiCRo [110] partitions a pretrained model into brain-inspired cognitive networks at the *intra-model* level; the Solver Contract operates at the *inter-agent* level. These are complementary: a Solver Tree could use MiCRo-style models as leaf solvers while composing them through typed contracts. Contract-bounded solvers may use different data, objectives, and reward signals when their parameters are isolated, unlike MoE experts trained through a shared loss. Meta-Harness [42] demonstrates that the orchestration layer around a model (prompts, tools, checking logic) is itself an optimizable artifact. This supports treating composition and execution harnesses as distinct design objects; it does not validate the Solver Contract itself.

Two older lineages provide conceptual precedents for the paper’s deepest claims. Hofstadter’s strange loop [111] is the classic account of structure that turns on and re-enters itself—the property Section 2.7 claims for intelligence when the decomposition algorithm is applied to its own target. Levin’s multi-scale competency architecture [29] documents a biological precedent for the compositional claim of Section 1.1: collectives—cells, tissues, organs—that present as single agents to the level above, with goals and problem-solving competence at every scale. Neither provides empirical support that machine decompositions converge or compound.

10.4 Generator-Verifier Pipelines and Multi-Agent Training

MALT [65] trains a three-agent pipeline with credit assignment, achieving 15.66% improvement on MATH benchmarks. Multi-agent debate [69, 70] captures the intuition that opposing perspectives improve reasoning, though Smit et al. [112] demonstrate that consensus often regresses to token-level averaging. This paper’s proposed extensions are to make the structure fractal (each verifier can recursively decompose), bound depth with the marginal-value rule, and let non-compensatory gates return typed requests for upstream reframing. Among recursive architectures, ReDel [113] implements dynamic delegation trees; Meyerson et al. [114] demonstrate million-step completion. Both implement recursive decomposition with implicit handoffs; the Solver Tree adds typed contracts with acceptance gates at every boundary.

10.5 Outcome-Grounded Learning

Several consequential results in machine learning ground in *outcomes* rather than in human approval of outputs. AlphaFold [115] fits its representation to real protein structures; AlphaGeometry [116] and AlphaProof, to proof-checkers; AlphaZero [117], to game results; reinforcement learning from unit tests, to code that runs. The *referent* of the feedback is whether the result was right, not whether a rater liked reading it—an axis distinct from, and complementary to, the human-preference loop (RLHF) that aligns models to approval. In these examples the grounding mechanism is shaped to one domain and does not itself provide cross-domain conceptual reuse; the resulting grounding remains domain-bound.

The paper assumes access to outcome grounding and asks whether grounded structures can be composed and transferred across domains. Its architectural bet is that a shared conceptual layer may remain useful across domains rather than being rebuilt for each one. The reviewed examples do not supply a common mechanism for composing grounded structures across domains. The concept-context separation developed in Section 11.6 identifies the proposed locus of transfer. Architecturally, this loop follows preference alignment: a human aligns the base model, after which the contract network grounds that aligned but outcome-blind substrate in domains with fast external oracles such as code, molecules, forecasts, and proofs.

10.6 Adaptive Compute Allocation

Adaptive Computation Time [118], Mixture-of-Depths [119], and compute-optimal test-time strategies [120] allocate compute across tokens or layers. CATTS [121] applies confidence-aware scaling to agentic tasks. AVA [122] uses value-of-information-guided search—the closest existing work reviewed here, achieving 20–40% cost reduction within a single reasoning process. The marginal-value rule extends the principle across a tree of contract-bounded solvers. The proposed distinction is to pair vertical allocation with a lateral trigger when optimization stalls (Section 3.4) and with explicit hard seams that concentrate depth at boundaries whose failure collapses the plan; these are not central mechanisms in the adaptive-computation approaches reviewed here. Supporting evidence for treating topology as a design variable comes from recent work on scaling multi-agent collaboration [123], where irregular DAG topologies outperform regular ones as agent count increases, and from NetSafe [124], which demonstrates that network structure affects multi-agent vulnerability.

10.7 Structural Safety

AI safety research has focused on aligning individual model behavior through training. Dalrymple et al. [125] propose a Guaranteed Safe AI framework combining world models, safety specifications, and verifiers, but at the single-system level. Seshia et al. [126] advocate assume-guarantee contracts for compositional verification but note that such a theory “does not yet exist for AI-based systems.” Information-flow control for AI agents [127] provides structural security through typed information labels. This paper contributes a specific synthesis: constraint monotonicity (child constraints must be a superset of parent constraints), typed acceptance gates at trust boundaries, and the principle that some deceptive behavior must materialize across a typed boundary where it may be inspected. The topology can make safety obligations more explicit and locally auditable, but it cannot replace model-level safety or detect malicious behavior that satisfies the stated contract—paralleling capability-based security [128], where processes can only attenuate capabilities, never amplify them.

11 Limitations and Open Challenges

The architecture makes strong claims, and the evidence remains limited. The principal contribution is architectural: like Design by Contract [37], which formalized interface discipline for software modules, this paper proposes the Solver Contract as a structural primitive and develops its consequences. The mandatory-abstraction construction, experimental reference implementation, and companion studies [86, 2, 41] exercise partial mechanisms rather than provide full validation at scale.

11.1 The Bitter-Lesson Objection

The most formidable objection to this architecture is historical. Sutton’s bitter lesson observes that general methods leveraging computation—search and learning—have repeatedly displaced human-engineered structure: hand-crafted features, grammar-based parsing, expert systems [129]. Through that lens, typed contracts, four-test carves, and explicit solver trees appear as candidates for the same graveyard: scaffolding that the next model generation renders obsolete. The objection deserves its strongest formulation: if end-to-end learning at scale can internalize the functions this structure provides, then the architecture is a temporary crutch bearing permanent overhead.

The first answer is that the bitter lesson indicts human-authored *content*, whereas the human-engineered layer here is intentionally thin. What people supply is a pay-for-what-you-need coordination interface—at its mandatory core, two surfaces, Manifest and Execute: describe and invoke (Section 4). The remaining surfaces are priced per seam, each bought to serve an explicit coordination demand—trust, clarification, or learning—rather than domain theory. The content the bitter lesson targets—the carves, the routes, the topologies—is model-generated, and is intended to improve as the models improve. Even the four-test operator is an initial discipline rather than an immutable theory: because the architecture accommodates alternative search operators (§2), the operator itself remains subject to the same selection as the structures it generates. In this sense the proposal is closer to a protocol than to a repository of domain knowledge, and scale did not render the internet’s protocols obsolete; it runs on them. The architecture is accordingly exposed to scale on the upside: stronger base models would mean better carves, sharper gate judgments, and a richer archive.

The second answer distinguishes regimes. Scale confers the conventional decomposition for free: a model trained on human corpora absorbs institutional categories alongside knowledge—the path-dependence this paper begins from. Where fast external oracles exist—code, proofs, molecules—end-to-end learning may indeed internalize better carves on its own, and there the architecture’s residual claims are composability, auditability, and persistence rather than superior capability. Where no oracle exists, gradient pressure reinforces the inherited carve—precisely the inertia an explicit decomposition discipline is designed to resist; the bet that fresh carves beat inherited ones at matched compute is the first of the decisive tests already stated, not an added claim.

The third answer is that the architecture’s end product aligns with the lesson rather than opposing it. Section 8.4 proposes that the archive’s accumulated derivations amortize the decomposition faculty into weights: the structure is scaffolding that compiles into subsequent model generations—a curriculum generator for scale, not a competitor to it. The objection then survives in one falsifiable form: if a monolithic model trained at matched compute without the derivation archive equals both the derivation-trained model and the physical solver tree in carve quality and cross-domain reuse, then the explicit structure incurred overhead without conferring direction, and the bitter lesson holds here as well.

11.2 Trigger Heuristics and Evaluation Grounding

Two questions the architecture does not fully answer are when a rejection should trigger structural reframing rather than retry, and how to measure the value of a solution on open-ended problems. Both have architectural answers but not algorithmic guarantees.

For the first, the insolubility signatures of Section 5.1 require a pattern of failure rather than a single rejection. Their thresholds are not universally calibrated, however: they are tuned per domain, and the contract does not specify that tuning process.

For the second, Section 6.10 separates the empirical layer of a result (measurable outcomes: cost, completion, constraint satisfaction, downstream gate rejection rates) from the normative layer (whether the result was *good* in a way that matters). The empirical layer is objective and automatable. The normative layer bottoms out in human judgment on open-ended problems and in domain-specific ground truth on verifiable ones. This grounding is not algorithmically decidable and requires external anchoring.

11.3 The Compounding Hypothesis

The mandatory-abstraction construction mechanically illustrates persistence and structural reuse (Section 9), but not the stronger claim that reuse improves a Solver. It records 500 non-forced reuses against 286 creations, yet contains no outcome feedback, acceptance-gate learning, or Pathway Memory. Those counts show that a model can route through retained structure; they provide no learning curve and do not demonstrate per-invocation improvement or compounding.

The companion study implements and documents a population-search controller compatible with part of the proposed Discovery Protocol; it neither implements nor causally tests the full protocol or the Solver architecture [2]. It nevertheless supplies positive descriptive evidence for the representation claim. Its three graph conditions ended with curator-authored typed hierarchies containing 35, 49, and 34 mechanism nodes and visibly different root counts, depths, and branching. Across 75 planned graph positions, seven captured complete rejection–diagnosis–mechanism-changing–retry–admission chains; 40 of 46 final non-leaf category titles were visible in a later Explorer context. These measurements show executed, revisable, structured population state rather than a list of isolated winners.

They do not establish the older, stronger comparison that gating itself made the underlying solution space more structured than ungated search. Only the graph conditions received the ontology apparatus, each condition contributed one path-dependent trajectory, and the conditions differed in memory, roles, turns, tool use, and selection without a matched no-control cell. The study therefore does not isolate creativity, framing, utility, or search-efficiency gains. Better framing, better solutions, and compounding remain open.

The representation claim also carries a falsifier of its own. If the architecture’s value is that it supplies decomposition that human practice never wrote down, it should help *most* where expertise is tacit and undocumented—apprenticeship-heavy, slow-feedback, institution-bound domains—and measurably *least* where reasoning is already explicit, as in formal proofs or documented procedures. Comparable gains across both settings would support generic scaffolding rather than the distinctive missing-structure claim, even if the scaffolding remained useful.

The risk is specific: crystallized patterns may become frozen procedures that snap in novel contexts rather than flexible scaffolding that adapts. If a Sema pattern encoding “distribution fairness” transfers from healthcare allocation to disaster-relief logistics, the ratchet works. If it silently encodes assumptions specific to healthcare that fail in disaster-relief contexts, the pattern is not reusable reasoning infrastructure; it is a brittle macro with a cryptographic seal. The architecture provides two defenses: the marginal-value rule detects when an imported pattern is not reducing uncertainty, and the lateral jump triggers reframing. Whether these suffice for cross-domain compounding at scale remains the central open question.

The risk compounds as well as the capability. A flawed node that clears its gates may attract traffic; traffic supplies more feedback, specialization, reputation, and routing priority; cross-domain reuse can then turn one local error into correlated failures. Content addressing guarantees stable identity, not truth. Evaluation must therefore measure contamination and recovery—how quickly a bad highway is detected, quarantined, deprecated, rolled back, or bypassed—and whether independent routing ecosystems limit common-mode failure.

A falsification instrument for this hypothesis follows once Pathway Memory and acceptance gates are instrumented: track, over a training run, whether general nodes stay unified and accrue cross-domain traffic, or fragment into domain-specific children. Persistent generality would be positive evidence for cross-domain compounding; progressive fragmentation would indicate thin transfer. The present construction cannot perform this test because it has no learning curve, and its static endpoint already shows structural fragmentation: 187 of 286 problem-created nodes remain single-case and 36 of 59 new abstraction parents serve one case (Section 9.2).

11.4 Scaffold Dependence and Independent Reproduction

The canonical construction supplies only RootSolver as authored persistent graph state, correcting the unintended 29-node initialization of the superseded Gemini artifact (Section 9.3). It is nevertheless not ontology-unprimed. Its bootstrap call receives the paper’s four tests, examples, and proposed General Problem-Solving Protocol, then derives 24 nodes before seeing the corpus. Its topology is model-authored and problem-blind but still theory-conditioned. Structural reuse inside that one graph cannot establish that independent systems would find the same concepts.

The reproduction study therefore requires independent Root-only constructions under several prompt conditions and two scores. *Invariant convergence* asks which high-traffic nodes or subgraphs recur across models, random seeds, and problem orders when no graph is shared. Equivalence must be semantic—matching constituents, bounds, and probe routing—rather than lexical. *Complementary utility* asks whether divergent carves improve blind outcomes, coverage, robustness, or uncertainty reduction beyond convergent carves and a null baseline of arbitrary divergence. A different graph counts as discovery only when its difference helps.

Matched ablations should compare the full theory-conditioned operator with Root-only constructions that remove the upward-first instruction, omit the four tests and examples, scramble the proposed decomposition, or substitute an equally abstract alternative scaffold. If these controls attract comparable reuse and produce comparable outcomes, the current artifact has shown flexible model-authored organization rather than leverage from the proposed method. If the complete operator wins under matched model, memory, compute, and corpus order, the comparison begins to support the stronger claim.

Content-addressing supplies reuse by *adoption*: once a Solver specification is adopted by Sema hash, every caller shares the identity of that specification without rediscovery. Independent convergence and complementary utility concern reuse by *discovery*. The architecture supports both forms while retaining divergent structures for later evaluation. A further axis is strategy-level reuse—retaining methods such as constraint inversion or stress-testing a cut rather than only their conceptual outputs—for which the companion Sema protocol provides the substrate [41].

11.5 The RootSolver Strategy

Because the RootSolver sees every problem enter the system (Section 4.3), its routing decisions determine which highways densify. The open problem is evaluation: when several routes could handle one problem, how does it learn which produces better outcomes?

Consider a concrete case. “Plan the best possible birthday party” could route to a CreationSolver (which decomposes into interpretation, emotional design, ideation, and logistics) or to a ConsultingSolver (which asks twenty structured questions and produces a checklist). Both return status: success. Both pass their gates. But the outcomes may differ profoundly, and the quality signal must be *independent of the decomposition path*. The proposed OutcomeArbiterSolver (Section 6.10) supplies one mechanism: blind evaluation behind a hard seam. Three difficulties remain.

Normative judgment resists objectification. For open-ended problems (“best possible birthday,” “design an economic system”), whether the result was good is irreducibly subjective. *Credit assignment between route and execution is hard*. A brilliant route with mediocre leaf solvers produces a mediocre result; a mediocre route with brilliant leaves may succeed anyway. The OutcomeArbiterSolver scores the final result, but the RootSolver needs to know whether the *route* was responsible. *The cold-start problem reappears at the root*. Comparative execution—routing the same problem through multiple paths—produces the cleanest signal but is the most expensive. The RootSolver that most needs good routing data is the one that has none.

These difficulties converge on a prediction: the RootSolver’s Pathway Memory will improve fastest for well-specified, frequently recurring problem classes and slowest for novel, open-ended ones.

11.6 The Concept–Context Split

For a conceptual Solver to operate across domains, the concept must be separated from the domain-specific context. This raises a precise question: what is hashed and persisted (the conceptual structure) versus what varies per invocation (the domain-specific payload)?

An UncertaintyPropagationSolver has an operational specification: its decomposition into sub-concepts, acceptance gates, and routing patterns. A Sema hash identifies that specification. The payload—weather data vs. financial data—varies per invocation. The specification persists; the data changes per call. This is precisely how a function works: the function definition is reusable; the arguments change.

The question is where the concept–context boundary falls. If too much domain context is required for the concept to operate effectively, the Solver becomes domain-specific and the highway never forms. If too little context is passed, the result is abstract but useless. Finding the right boundary is a design challenge the architecture does not automatically solve.

There is a deeper reason this boundary is the crux rather than a detail. Grounding a representation in a domain’s feedback is the act of fitting it to that domain—so grounding *specializes whatever layer absorbs the feedback*. In the outcome-grounded systems reviewed above, the grounding mechanism remains domain-shaped rather than supplying a portable conceptual core (Section 10). The concept–context separation is the architecture’s bet against this: a typed bottleneck might direct domain-specific feedback into the context adapter (the upstream translation from domain payload to abstract variables) while the core sees only the abstract variables, so grounding specializes the adapter and spares a portable core. The failure mode is a *leaky seam*: if domain specifics bleed into the core, grounding consumes it and the highway never forms.

This sharpens the decisive experiment. Ground a concept-solver where the outcome-oracle is fast and clean—code that runs, a molecule that binds, a forecast that resolves—then import it to a domain whose oracle is slow, and measure whether the grounding survives the move: does the imported core improve the new domain, or did grounding specialize it after all? That is the cross-domain compounding hypothesis in its most testable form, and whether the concept–context seam confines grounding’s specialization to the adapter is the structural question on which it turns.

11.7 Context Fragmentation

For highly entangled problems where the solution relies on holistic, unarticulated understanding, forcing the problem across a typed boundary can destroy the information needed to solve it. For tightly coupled tasks where a single powerful model can hold the entire context, a monolithic approach may outperform. Meta Protocols (Section 6.10) provide a partial response: meta-level solvers may preserve some global context, but only in lossy, approximate form.

A candidate mitigation is a *root-intent skip-connection*: every leaf node receives, in addition to its strict typed Task, a read-only pointer or low-dimensional representation of the original RootSolver Manifest. A soft-prompt or cross-attention mechanism could condition the leaf on this representation without passing the full root prompt through every intermediate Solver. The structural analogue is a residual connection in a deep network: a thin trace of the original intent bypasses intermediate boundaries. Whether such a trace preserves useful global context without reintroducing instruction leakage remains to be tested.

11.8 The AcceptSpec Generation Problem

Generating rigorous gate criteria is itself a difficult problem, especially for open-ended tasks. If the parent produces a leaky or impossible AcceptSpec, the child may deadlock or pass vacuous output. The Consult surface allows criteria to be negotiated before execution, and the override-with-documentation mechanism provides an escape from inadequate criteria (Section 4.1), but neither resolves the fundamental difficulty of specifying what “good” means for open-ended problems.

11.9 The Estimation Cold-Start

The marginal-value rule requires accurate estimates for expected improvement ($\mathbb{E}[V]$) and cost (C). For well-mapped problem classes, Pathway Memory grounds these in historical data. For genuinely novel classes, no precedent exists and the rule degrades to a heuristic exploration budget. A practical mitigation is to formalize a *probe phase*: when a node detects a zero-shot environment, it spawns lightweight, low-cost probe Solvers tasked with mapping the problem space and establishing baseline cost-to-improvement ratios. Once probes populate initial Pathway Memory, the orchestrator can transition to deeper execution with better estimates. This trades initial estimation uncertainty for additional exploration cost and latency.

11.10 Adversarial Dynamics

Explicit acceptance gates are vulnerable to Goodhart’s Law: upstream Solvers may optimize for mechanical satisfaction of tests rather than semantic intent. *Formatting sycophancy*—outputs satisfying schema while remaining semantically hollow—combined with evaluator collusion (Section 4.1) creates a failure mode where compliant but empty outputs pass unchallenged. The mitigations (cross-family ensemble juries, red-team solvers) increase inference costs. Determining the minimal adversarial redundancy to prevent mode collapse at hard seams remains open. One possible economic mechanism is cryptoeconomic staking with slashing for verified plausible-garbage payloads, converting a denial-of-service economics problem into a bonded one.

11.11 Orchestration Overhead

Every seam that bites introduces friction. The contract demands typed serialization, content-identity checks for the AcceptSpec, substantive gate evaluation, prompt hydration, and state management at every node. For high-velocity, low-stakes environments, overhead may outweigh benefit. The Sema vocabulary operationalizes one proposed mitigation via the `OptimisticSolver#ae61` pattern, which mandates single-turn execution with post-action correction rather than pre-action verification; the companion study reports lower coordination latency under a related protocol condition [41]. Deep trees still risk context-window bloat; content-addressed pointers to shared artifact stores can mitigate payload duplication, but managing state consistency across asynchronous branches introduces distributed-systems complexities. A direct comparison of total cost versus monolithic hidden chain-of-thought (e.g., OpenAI o-series, Gemini Deep Think [130]) remains future work.

11.12 Empirical Validation Gaps

The companion studies exercise specific precursors on the deployment spectrum (Section 4.3): pattern-enriched dialogue (Level 2) stabilized coordination latency in the reported trading swarms [41], while the Ontology study incorporated previously released purpose-separated Orthogonal Insight artifacts and implemented curator-driven redirection and category revision [1, 2]. Neither study instantiates or causally validates the full contract stack. Continuous validation across the spectrum

remains sparse, and empirical data is needed to identify where formalization yields its largest marginal returns. Current evidence relies on synthetic domains; real-world deployment across highly entangled systems is required to test graceful degradation under noise.

The matched-compute test must also be a matched-memory test. The conventional baselines—a single frontier model and a role-based agent team—receive the same global retention of evaluated traces, outcome feedback, and retrieval budget as the conceptual system; otherwise the experiment confounds decomposition with memory. Every orchestration call, gate evaluation, and re-carve is charged. Four ablations isolate the mechanism: the contract over a deliberately scrambled decomposition; the task-organized team upgraded with typed gates; a Root-only construction without the upward-first and four-test operator; and independent repetitions across models, prompts, and problem orders to test convergence. On verifiable problems the outcome measure is objective—the proof checks, the code passes its tests, the forecast resolves; on open-ended ones a blind `OutcomeArbiterSolver` scores final artifacts without sight of their architecture. The sample should span tacit and already-explicit reasoning domains, where the theory predicts different gains.

12 Conclusion

The paper advances two commitments: retain evaluated solution structure for reuse, and re-decompose problems without assuming inherited human categories provide the best cut. A fresh carve can change which candidate mechanisms become visible and composable, potentially opening solution families an inherited framing did not make salient. The five-surface contract, marginal-value rule, Pathway Memory, and content-addressed commons operationalize these commitments. Recursive encapsulation is core: the root bounds the problem space; each concrete case climbs through adjacent specialization parents and returns through the exact route; each composite Solver synthesizes differentiated children into one outward capability; and an unchanged child may reappear beneath multiple parents through different edge-local roles. The same mechanisms let the carve deepen, branch, and remain revisable.

The ten protocols make the proposal concrete and remain useful whether or not independent runs converge on a common ontology; they do not establish that the contract is sufficient. Companion studies report stabilized coordination latency in protocol-bound trading swarms and an ontology-governed curator that redirected one retry and revised a candidate map in another trace. Neither result isolates the full architecture’s causal benefit, which still requires the controls above.

Typed boundaries can catch errors before propagation, turn rejection into a signal to reframe rather than retry, and generate local learning signals: Pathway Memory records gate feedback while Sema preserves reusable structure. This cycle is possible, not inevitable. Its tests are whether fresh carves improve outcomes under matched compute and memory, independent carves recover recurring invariants, divergent carves add complementary value beyond arbitrary difference, and reuse subsequently compounds across domains.

The central claim concerns leverage rather than a replacement for stronger models. As the scientific method let discoveries accumulate and correct one another, *fractal intelligence* denotes the broader possibility that machine reasoning might do the same with solution structures; the Solver architecture develops one possible realization. Whether such a network distributes cognitive power or concentrates it depends on how, and by whom, its goals, gates, routing, and memory are governed. The open question is whether cumulative, recursively restructuring intelligence can operate globally and build an internet of reasoning from structures that survive use.

Code and Artifact Availability

Paper source and the complete public mandatory-abstraction construction artifact—prompt, controller, schema, audit, structured model outputs, state, and interactive viewer, but no private reasoning traces—are available at <https://github.com/emergent-wisdom/fractal-intelligence>. The superseded seeded Gemini artifact is retained there only for reproducibility and error provenance. The companion *Fractal Intelligence Protocol* v0.2.0 revision—an experimental reference implementation of selected Solver-architecture and reviewed-topology mechanisms—is maintained at <https://github.com/emergent-wisdom/fractal-intelligence-protocol>. The paper’s dated release history is archived at [doi:10.5281/zenodo.19462645](https://doi.org/10.5281/zenodo.19462645).

References

- [1] Henrik Westerberg. Algorithmic creativity via strange worlds: A multi-agent toolkit for escaping the median trap. Zenodo, 2025. Version 1; publication date 8 December 2025.
- [2] Henrik Westerberg. The ontology of the alien: Escaping the median trap in LLM ideation. Zenodo, 2026.
- [3] Stuart A. West, Roberta M. Fisher, Andy Gardner, and E. Toby Kiers. Major evolutionary transitions in individuality. *Proceedings of the National Academy of Sciences of the United States of America*, 112(33):10112–10119, 2015.
- [4] Richard E. Michod. Evolution of individuality during the transition from unicellular to multicellular life. *Proceedings of the National Academy of Sciences of the United States of America*, 104(suppl. 1):8613–8618, 2007.
- [5] Henrik Westerberg. Entangled alignment: When safety is the substrate. Preprint, 2026.
- [6] Henrik Westerberg. Understanding graph: Persisting the invisible thinking. Preprint, 2026.
- [7] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011.
- [8] Tiansheng Huang, Sihao Hu, Fatih Ilhan, Selim Furkan Tekin, Zachary Yahn, Yichang Xu, and Ling Liu. Safety tax: Safety alignment makes your large reasoning models less reasonable. *arXiv preprint arXiv:2503.00555*, 2025.
- [9] Allen Newell and Herbert A. Simon. *Human Problem Solving*. Prentice-Hall, 1972.
- [10] Charles Babbage. *On the Economy of Machinery and Manufactures*. Charles Knight, London, 1832. Source of the Babbage Principle regarding the division of mental labor.
- [11] Herbert A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.
- [12] Marvin Minsky. *The Society of Mind*. Simon & Schuster, 1986.
- [13] David Lorge Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [14] W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, 1956.

- [15] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.
- [16] Hans-Jürgen Warnecke. *The Fractal Company: A Revolution in Corporate Culture*. Springer, Berlin, Heidelberg, 1993.
- [17] Kwangyeol Ryu, Youngho Son, and Mooyoung Jung. Modeling and specifications of dynamic agents in fractal manufacturing systems. *Computers in Industry*, 52(2):161–182, 2003.
- [18] Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980.
- [19] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [20] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Language agents as optimizable graphs. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.
- [21] Qing Ye and Jing Tan. Agent contracts: A formal framework for resource-bounded autonomous AI systems. *arXiv preprint arXiv:2601.08815*, 2026.
- [22] Fritz Zwicky. *Discovery, Invention, Research Through the Morphological Approach*. Macmillan, New York, 1969.
- [23] Yash Raj Singh. Graph-memoized reasoning: Foundations structured workflow reuse in intelligent systems. *arXiv preprint arXiv:2511.15715*, 2025.
- [24] Barbara Minto. *The Pyramid Principle: Logic in Writing and Thinking*. Pitman, 1985.
- [25] Aristotle. *Categories*. 350. Translated by J. L. Ackrill. In *The Complete Works of Aristotle*, edited by Jonathan Barnes, vol. 1, Princeton University Press, 1984.
- [26] Rudolf Wille. Restructuring lattice theory: An approach based on hierarchies of concepts. In Ivan Rival, editor, *Ordered Sets*, volume 83 of *NATO Advanced Study Institutes Series*, pages 445–470. Reidel, Dordrecht, 1982.
- [27] Eleanor Rosch. Principles of categorization. In Eleanor Rosch and Barbara B. Lloyd, editors, *Cognition and Categorization*, pages 27–48. Lawrence Erlbaum Associates, Hillsdale, NJ, 1978.
- [28] Beáta Oborny. The plant body as a network of semi-autonomous agents: A review. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 374(1774):20180371, 2019.
- [29] Michael Levin. Technological approach to mind everywhere: An experimentally-grounded framework for understanding diverse bodies and minds. *Frontiers in Systems Neuroscience*, 16:768201, 2022.
- [30] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.

- [31] Eric L. Charnov. Optimal foraging, the marginal value theorem. *Theoretical Population Biology*, 9(2):129–136, 1976.
- [32] Stuart Russell and Eric Wefald. *Do the Right Thing: Studies in Limited Rationality*. Artificial Intelligence. MIT Press, Cambridge, MA, 1991.
- [33] Samuel J. Gershman, Eric J. Horvitz, and Joshua B. Tenenbaum. Computational rationality: A converging paradigm for intelligence in brains, minds, and machines. *Science*, 349(6245):273–278, 2015.
- [34] C. Nicolò De Sabbata, Theodore R. Sumers, Badr AlKhamissi, Antoine Bosselut, and Thomas L. Griffiths. Rational metareasoning for large language models. *arXiv preprint arXiv:2410.05563*, 2024. NeurIPS 2024 Workshop.
- [35] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [36] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 583, 1969.
- [37] Bertrand Meyer. Applying “Design by contract”. *Computer (IEEE)*, 25(10):40–51, 1992.
- [38] Barbara H. Liskov and Jeannette M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.
- [39] Robert Bruce Findler and Matthias Felleisen. Contracts for higher-order functions. In *Proceedings of the Seventh ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 48–59. ACM, 2002.
- [40] Jerome H. Saltzer, David P. Reed, and David D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.
- [41] Henrik Westerberg. Sema: When the hash is the word. Preprint, 2026.
- [42] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- [43] Keith E. Stanovich and Richard F. West. Individual differences in reasoning: Implications for the rationality debate? *Behavioral and Brain Sciences*, 23(5):645–726, 2000.
- [44] Jonathan St. B. T. Evans. Dual-processing accounts of reasoning, judgment, and social cognition. *Annual Review of Psychology*, 59:255–278, 2008.
- [45] Jonathan W. Schooler, Stellan Ohlsson, and Kevin Brooks. Thoughts beyond words: When language overshadows insight. *Journal of Experimental Psychology: General*, 122(2):166–183, 1993.
- [46] Ap Dijksterhuis and Teun Meurs. Where creativity resides: The generative power of unconscious thought. *Consciousness and Cognition*, 15(1):135–146, 2006.
- [47] Harold Pashler. Dual-task interference in simple tasks: Data and theory. *Psychological Bulletin*, 116(2):220–244, 1994.

- [48] Akash Gupta, Ivaxi Sheth, Vyas Raina, Mark Gales, and Mario Fritz. LLM task interference: An initial study on the impact of task-switch in conversational history. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [49] Leyang Shen, Gongwei Chen, Rui Shao, Weili Guan, and Liqiang Nie. MoME: Mixture of multimodal experts for generalist multimodal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [50] Matt Keon, Aabid Karim, Bhoomika Lohana, Abdul Karim, Thai Nguyen, Tara Hamilton, and Ali Abbas. Galton’s law of mediocrity: Why large language models regress to the mean and fail at creativity in advertising. *arXiv preprint arXiv:2509.25767*, 2025.
- [51] Tony Mason. Arbiter: Detecting interference in LLM agent system prompts—a cross-vendor analysis of architectural failure modes. *arXiv preprint arXiv:2603.08993*, 2026.
- [52] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.
- [53] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [54] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [55] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [56] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [57] Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011.
- [58] Henrik Westerberg. Temporal hindsight learning: Blindness as teacher, hindsight as curriculum. Preprint, 2026.
- [59] David Hume. *A Treatise of Human Nature*. John Noon, London, 1739–1740. Book III, Part I, Section I: the is-ought distinction.
- [60] Nick Bostrom. *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press, 2014.
- [61] Robin Hanson. Shall we vote on values, but bet on beliefs? *Journal of Political Philosophy*, 21(2):151–178, 2013.
- [62] Joyce E. Berg, Forrest D. Nelson, and Thomas A. Rietz. Prediction market accuracy in the long run. *International Journal of Forecasting*, 24(2):285–300, 2008.
- [63] Abraham Othman and Tuomas Sandholm. Decision rules and decision markets. In *Proceedings of the 9th International Conference on Autonomous Agents and Multiagent Systems*, pages 625–632, 2010.

- [64] Yiling Chen, Ian A. Kash, Michael Ruberry, and Victor Shnayder. Eliciting predictions and recommendations for decision making. *ACM Transactions on Economics and Computation*, 2(2):6:1–6:27, 2014.
- [65] Sumeet Ramesh Motwani, Chandler Smith, Rocktim Jyoti Das, Rafael Rafailov, Ivan Laptev, Philip H. S. Torr, Fabio Pizzati, Ronald Clark, and Christian Schroeder de Witt. MALT: Improving reasoning with multi-agent LLM training. *arXiv preprint arXiv:2412.01928*, 2024.
- [66] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [67] Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [68] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.
- [69] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.
- [70] Tian Liang, Zhiwei He, Wenxiang Jiao, et al. Encouraging divergent thinking in large language models through multi-agent debate. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [71] Rohan Anil, Gabriel Pereyra, Alexandre Passos, Robert Ormandi, George E Dahl, and Geoffrey E Hinton. Large scale distributed neural network training through online distillation. *arXiv preprint arXiv:1804.03235*, 2018.
- [72] Ying Zhang, Tao Xiang, Timothy M Hospedales, and Huchuan Lu. Deep mutual learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [73] DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence, 2026. Post-training trains domain experts independently (SFT + GRPO), then consolidates them into one student via on-policy distillation (§5.1 of the report).
- [74] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- [75] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- [76] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–40, 2022.
- [77] Sarthak Mittal, Yoshua Bengio, and Guillaume Lajoie. Is a modular architecture enough? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [78] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. From sparse to soft mixtures of experts. In *International Conference on Learning Representations (ICLR)*, 2024.

- [79] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [80] Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. Stream of Search (SoS): Learning to Search in Language, 2024. Trains on full search trajectories, dead ends and backtracking included.
- [81] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The Curse of Recursion: Training on Generated Data Makes Models Forget, 2023.
- [82] Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E. Gonzalez, and Bin Cui. Buffer of Thoughts: Thought-Augmented Reasoning with Large Language Models. *Advances in Neural Information Processing Systems*, 37, 2024. Retrieves distilled thought templates at inference; arXiv:2406.04271.
- [83] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017.
- [84] Peter R. Huttenlocher. Synaptic density in human frontal cortex—developmental changes and effects of aging. *Brain Research*, 163(2):195–205, 1979.
- [85] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2018.
- [86] Henrik Westerberg. Fractal Intelligence Protocol: Experimental coordinator and node SDK for the Solver Contract. GitHub repository, 2026. Version 0.2.0; source revision f6b4aa844f52.
- [87] Peter Gärdenfors. *Conceptual Spaces: The Geometry of Thought*. MIT Press, Cambridge, MA, 2000.
- [88] Edsger W. Dijkstra. On the role of scientific thought. *Selected Writings on Computing: A Personal Perspective*, pages 60–66, 1982. Original manuscript dated 1974. Introduces the term “separation of concerns.”.
- [89] Kutluhan Erol, James Hendler, and Dana S. Nau. HTN planning: Complexity and expressivity. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94)*, pages 1123–1128, 1994.
- [90] Dana S. Nau, Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman. SHOP2: An HTN planning system. *Journal of Artificial Intelligence Research*, 20:379–404, 2003.
- [91] Yanming Liu, Xinyue Peng, Zixuan Yan, Yanxin Shen, Wenjie Xu, Yuefeng Huang, Xinyi Wang, Jiannan Cao, Jianwei Yin, and Xuhong Zhang. Dep-search: Learning dependency-aware reasoning traces with persistent memory. *arXiv preprint arXiv:2601.18771*, 2026.
- [92] Songyang Gao, Yuzhe Gu, Zijian Wu, Lingkai Kong, Wenwei Zhang, et al. Long-horizon reasoning agent for olympiad-level mathematical problem solving. *arXiv preprint arXiv:2512.10739*, 2025.

- [93] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021.
- [94] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [95] Douglas B. Lenat. CYC: A large-scale investment in knowledge infrastructure. *Communications of the ACM*, 38(11):33–38, 1995.
- [96] Ian Niles and Adam Pease. Towards a standard upper ontology. In *Proceedings of the 2nd International Conference on Formal Ontology in Information Systems (FOIS)*, pages 2–9, 2001.
- [97] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM’94)*, pages 456–463, 1994.
- [98] Foundation for Intelligent Physical Agents. FIPA ACL message structure specification. Standard SC00061G, 2002. IEEE Foundation for Intelligent Physical Agents.
- [99] Anthropic. Model context protocol. Anthropic, 2024. Open protocol for LLM-tool integration.
- [100] Google. Announcing the agent2agent protocol (A2A). Google Developers Blog, 2025.
- [101] Agent Network Protocol Community. Agent network protocol (ANP), 2025. Decentralized agent discovery and collaboration via DIDs and JSON-LD.
- [102] Peter Steinberger. OpenClaw: Personal AI assistant. <https://openclaw.ai>, 2026. Open-source autonomous agent framework with multi-agent routing and persistent execution.
- [103] Sirui Hong, Mingjie Zhuge, Jonathan Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [104] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.
- [105] Claudiu Leoveanu-Condrei. A DbC inspired neurosymbolic layer for trustworthy agent design. *arXiv preprint arXiv:2508.03665*, 2025.
- [106] Varun Pratap Bhardwaj. Agent behavioral contracts: Formal specification and runtime enforcement for reliable autonomous AI agents. *arXiv preprint arXiv:2602.22302*, 2026.
- [107] Zahra Moslemi, Keerthi Koneru, Yen-Ting Lee, Sheethal Kumar, and Ramesh Radhakrishnan. POLARIS: Typed planning and governed execution for agentic AI in back-office automation. *arXiv preprint arXiv:2601.11816*, 2026.
- [108] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [109] Robert E. Wray et al. Applying cognitive design patterns to general LLM agents. In *Artificial General Intelligence (AGI)*, 2025. [arXiv:2505.07087](https://arxiv.org/abs/2505.07087).

- [110] Badr AlKhamissi, C. Nicolò De Sabbata, Greta Tuckute, Zeming Chen, Martin Schrimpf, and Antoine Bosselut. Mixture of cognitive reasoners: Modular reasoning with brain-like specialization. *arXiv preprint arXiv:2506.13331*, 2025.
- [111] Douglas R. Hofstadter. *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books, 1979.
- [112] Andries Smit, Paul Duckworth, Nathan Grinsztajn, Thomas D Barrett, and Arnu Pretorius. Should we be going MAD? A look at multi-agent debate strategies for LLMs. In *International Conference on Machine Learning (ICML)*, 2024.
- [113] Andrew Zhu, Liam Dugan, and Chris Callison-Burch. ReDel: A toolkit for LLM-powered recursive multi-agent systems. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 162–171, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [114] Elliot Meyerson et al. Solving a million-step LLM task with zero errors, 2025. Empirical validation of the Babbage Principle via massively decomposed agentic processes.
- [115] John Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 2021.
- [116] Trieu H. Trinh et al. Solving olympiad geometry without human demonstrations. *Nature*, 625, 2024.
- [117] David Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362, 2018.
- [118] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [119] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.
- [120] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [121] Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Agentic test-time scaling for WebAgents. *arXiv preprint arXiv:2602.12276*, 2026.
- [122] anonymous. Anytime verified agents: Adaptive compute allocation for reliable LLM reasoning under budget constraints. *Transactions on Machine Learning Research (TMLR)*, 2025. Under review. OpenReview:JMDCMf7mlF.
- [123] Chen Qian et al. Scaling large-language-model-based multi-agent collaboration. In *ICLR*, 2025.
- [124] Yifan Zeng et al. NetSafe: Exploring the topological safety of multi-agent networks. *arXiv preprint*, 2025.
- [125] David Dalrymple, Joar Skalse, Yoshua Bengio, Stuart Russell, Max Tegmark, Sanjit A. Seshia, Steve Omohundro, et al. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.

- [126] Sanjit A. Seshia, Dorsa Sadigh, and S. Shankar Sastry. Toward verified artificial intelligence. *Communications of the ACM*, 65(7):46–55, 2022.
- [127] Manuel Costa, Boris Kopf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.
- [128] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [129] Richard S. Sutton. The Bitter Lesson. Incomplete Ideas (blog), March 2019.
- [130] Google DeepMind. Gemini 3. Model Card, 2026.